

4.1 mcal 項目間演算

c=パラメータで指定した計算式で計算をおこない、a=パラメータで指定した項目名に出力する。mcal が出力する項目は、プログラムの単純化のため、例外なく 1 つに限定している。計算式の詳細は後述の「式の構成要素」を参照のこと。

書式

```
mcal a= c= [i=] [o=] [-assert.diffSize] [-assert.nullkey] [-assert.nullin] [-assert.nullout] [-nfn] [-nfno] [-x] [-q]
[tmpPath=] [precision=] [--help] [--help1] [--version]
```

パラメータ

- a= 新たに計算結果の出力として追加される項目の名前を指定する。
- c= 用意された関数を組み合わせて計算する式を指定する。

利用例

以下に mcal を使った簡単な例を示す。個々の関数や演算子についての詳細は、それぞれ個別に解説を用意しているので、そちらを参照されたい。

```
# 入力ファイル (dat1.csv)
顧客, 数量, 単価
A,3,10
B,1,15
C,2,20

$ mcal c='${数量}*${単価}' a=金額 i=dat1.csv
顧客, 数量, 単価, 金額
A,3,10,30
B,1,15,15
C,2,20,40

$ mcal c='${数量}*${単価}<=30' a=金額 30 以下 i=dat1.csv
顧客, 数量, 単価, 金額 30 以下
A,3,10,1
B,1,15,1
C,2,20,0

$ mcal c='if(top(),${単価},#{}+${単価})' a=単価累計 i=dat1.csv
顧客, 数量, 単価, 単価累計
A,3,10,10
B,1,15,25
C,2,20,45
```

シェルでの利用時の注意事項

UNIX 系の OS で bash などのシェルを利用する場合、演算子などの記号はシェルにとっても特殊な意味を持つことが多い。例えば \$ 記号に続く単語列はシェル変数を意味する。一方で mcmd では項目値を参照するために \$ を使っている。そこでシェルにそれらの記号を解釈させないために、以下のようにシングルクォーツで囲む必要がある。

```
$ mcal c='${date}-10'
```

エラーメッセージ

```
#ERROR# unknown function or operator
```

このエラーメッセージが出るということは、関数もしくは演算子の指定に誤りがある。例えば、以下のような文字列の結合関数 `cat` についてのエラーメッセージについて考える。

```
$ mcal c='cat("-",1,2)'
ERROR : unknown function or operator: cat_SNN(cat_SN) (kgcal)
```

「`cat_SNN`」のアンダーバーの前の `cat` は関数名を示し、その後の `SNN` は引数の型を示している。`S` は文字列型、`N` は数値型、`D` は日付型、`T` は時刻型、`B` は真偽型である。3つの引数を指定しているので3文字 (`SNN`) となっている。すなわち、このエラーメッセージは「引数として `SNN` をとる `cat` という名前の関数」は登録されていないということの意味する。以下のように2,3番目の引数を文字列型にすればよい。

```
$ mcal c='cat("-", "1", "2")'
```

ここで、エラーメッセージの括弧の中は2文字 (`SN`) となっているが、それは2番目以降のパラメータが可変個指定可能であるため、それらの代表として1つだけ表記しているためである。

関連コマンド

msel : 演算の結果を用いて行選択するのであればこのコマンドを使う。

4.2 式の構成要素

`mcal` で記述する式を構成する要素は、定数、項目値、演算子、関数の4つに大きく分類できる。そして、それぞれの要素において、ユーザはデータ型を意識して利用する必要がある。`mcal` が扱うデータはCSVテキストなので、値は全て文字列として与えられており、それら文字列を `mcal` でどのようなデータ型として扱うかはユーザに委ねられているからである。`mcal` が規定するデータ型は文字列型 (*str*)、数値型 (*num*)、日付型 (*date*)、時刻型 (*time*)、論理型 (*bool*) の5つである。以下では、式を構成するそれぞれの要素で、5つのデータ型をどのように扱うかを示す。

4.3 定数

表 4.1 定数の書式一覧

データ型	書式	内容	例
数値 (<i>num</i>)	整数、実数の文字列表記	内部的には全て倍精度実数を利用	20, 0.55, 1.5*e10
文字列 (<i>str</i>)	"文字列"	ダブルクォーテーションで括った文字列	"abc" "日本語"
日付 (<i>date</i>)	0dyyyymmdd	先頭に"0d"を付けた年月日固定長	0d20080923
時刻 (<i>time</i>)	0tyyyymmddHHMMSS	先頭に"0t"を付けた年月日時分秒固定長	0t20080923121115
	0tHHMMSS	先頭に"0t"を付けた時分秒固定長 (内部的に本日の日付が補完される)	0t121115
論理 (<i>bool</i>)	0b1, 0b0	先頭に"0b"を付けた"1"(true)もしくは"0"(false)	0b1, 0b0

4.4 項目値

CSVデータ上の項目名を指定するには表 4.2 に示される通り、CSVデータをどのデータ型として扱うかによって異なってくる。CSVデータは全て文字列データであるために、それらをどのような型のデータとして扱うかはユーザの

判断に任されている。

表 4.2 項目値の書式一覧

データ型	書式	CSV データ内容	例
数値 (<i>num</i>)	<code>\${ 項目名 }</code>	整数、実数 (指数表現含む) の文字列表記	<code>\${amount}</code> , <code>\${株価}</code>
文字列 (<i>str</i>)	<code>\$s{ 項目名 }</code>	文字列	<code>\$s{gender}</code> , <code>\$s{性別}</code>
日付 (<i>date</i>)	<code>\$d{ 項目名 }</code>	年月日固定長 (yyyymmdd)	<code>\$d{date}</code> , <code>\$d{発注日}</code>
時刻 (<i>time</i>)	<code>\$t{ 項目名 }</code>	年月日時分秒の固定長 (yyyymmddHHMMSS) 時分秒 (HHMMSS) の固定長 (内部的に本日の日付が補完される)	<code>\$d{time}</code> , <code>\$d{出発時刻}</code>
論理 (<i>bool</i>)	<code>\$b{ 項目名 }</code>	項目値の一字目が"1"の時に true、"0"の時に false、 その他の場合には NULL と解釈される。	<code>\$b{condition}</code> , <code>\$b{条件}</code>

4.5 ワイルドカード

項目名にはワイルドカードを指定することができる。例えば `sum` 関数は複数の数値項目の合計を計算する関数であるが、ワイルドカードを指定することで、多数の項目を一つのワイルドカードで指定することも可能となる。例えば、入力データとして `A1,A2,A3` の3つの数値項目名があったとすると、`sum(${A*})` とすれば、`A1,A2,A3` の合計値を計算してくれる。もちろん `sum(${A*},${B*})` のように複数のワイルドカードを指定することも可能である。

4.6 前行の項目値

項目値の指定に `$` の代わりに `#` を指定すると、前行の項目値となる。ただし、先頭行は前行がないので NULL となる。各データ型における指定方法を 4.3 に示す。

表 4.3 前行の項目値の書式一覧

データ型	書式	例
数値 (<i>num</i>)	<code>#{ 項目名 }</code>	<code>#{amount}</code> , <code>#{株価}</code>
文字列 (<i>str</i>)	<code>#s{ 項目名 }</code>	<code>#s{gender}</code> , <code>#s{性別}</code>
日付 (<i>date</i>)	<code>#d{ 項目名 }</code>	<code>#d{date}</code> , <code>#d{発注日}</code>
時刻 (<i>time</i>)	<code>#t{ 項目名 }</code>	<code>#d{time}</code> , <code>#d{出発時刻}</code>
論理 (<i>bool</i>)	<code>#b{ 項目名 }</code>	<code>#b{condition}</code> , <code>#b{条件}</code>

4.7 前行の結果項目値

前行項目の指定において項目名を省略すると前行の計算結果項目の値となる。各データ型における指定方法を 4.4 に示す。

if 関数と `top()` 関数とを組み合わせる事で、累計計算などが可能となる。以下に、金額項目の累計計算例を示す。

```
$ mcal c='if(top(),${金額},${金額}+#{})' a=累計金額
```

表 4.4 前行の結果項目値の書式一覧

データ型	書式	例
数値 (<i>num</i>)	<code>#{ }</code>	<code>#{ }</code>
文字列 (<i>str</i>)	<code>#s{ }</code>	<code>#s{ }</code>
日付 (<i>date</i>)	<code>#d{ }</code>	<code>#d{ }</code>
時刻 (<i>time</i>)	<code>#t{ }</code>	<code>#d{ }</code>
論理 (<i>bool</i>)	<code>#b{ }</code>	<code>#b{ }</code>

4.8 算術演算子

+ や-などの算術演算子は数値型だけでなく、文字列型や日付型のデータに対しても定義されている。それらの一覧を表 4.5 に示す。

表 4.5 算術演算子一覧

演算子	書式	内容	例
加算 (+)	$num_1 + num_2$	数値の足し算	1.5+2.3 (3.8)
	$str_1 + str_2$	文字列の結合	"150"+"円" ("150 円")
	$date + num$	num 日後の日付	0d20121130+2 (0d20121202)
	$time + num$	num 秒後の時刻	0t095959+2 (0t100001)
減算 (-)	$num_1 - num_2$	数値の引き算	1.5-2.3 (-1.8)
	$str_1 - str_2$	部分文字列の削除 (貪欲マッチによる)	"aababa"-"a" ("bb") "aababa"-"aba" ("aba")
	$date - num$	num 日前の日付	0d20121202-2 (0d20121130)
	$time - num$	num 秒前の時刻	0t100001-2 (0t095959)
	$date_1 - date_2$	日付差	0d20121202-0d20121130 (2)
	$time_1 - time_2$	時刻差	0t095959-0t100001 (-2)
乗算 (*)	$num_1 * num_2$	掛け算	10*2 (20)
除算 (/)	num_1 / num_2	割り算	10/2 (5)
剰余 (%)	$num_1 \% num_2$	剰余	10%3 (1)
累乗 (^)	$num_1 ^ num_2$	累乗	10^3 (1000)

例の括弧内は結果を表す (内容は定数の表記で示している)。

4.9 比較演算子

比較演算は同一のデータ型の値同士でのみ適用可能である。全てのデータ型に共通した書式であり、以下では数値型についてのみ (例では文字型につても) 表 4.6 に示す。文字型、日付型、時刻型においても同様に利用できる。

表 4.6 比較演算子一覧

比較内容	書式	例
等しい	$num_1 == num_2$	1.5==1.5(0b1), "abc"=="abcd" (0b0)
等しくない	$num_1 != num_2$	1.5!=1.5(0b0), "abc"=="abcd" (0b1)
より大きい	$num_1 > num_2$	10>5(0b1), "abc">"abcd" (0b0)
より小さい	$num_1 < num_2$	10<5(0b0), "abc"<"abcd" (0b1)
以上	$num_1 >= num_2$	10>=10(0b1), "a">"" (0b1)
以下	$num_1 <= num_2$	8<=9(0b1), "a"<="a" (0b1)

例の括弧内は結果を表す (内容は定数の表記で示している)。

4.10 論理演算子

3つの論理演算子 (論理積、論理和、排他的論理和) が利用でき、それぞれの書式を表 4.7 に示す。また、それぞれの演算における真偽 (1:真,0:偽) の組み合わせとその結果を表 4.8, 表 4.9, 表 4.10 に示す。

4.11 演算子の優先順位

演算子 (後述) の優先順位は表 4.11 に示すとおりである。同一の演算子間の優先順位は出現順序による。優先順位を変更するときは括弧を利用すれば良い。

4.12 関数

以下では、数値関連 (4.12)、三角関数関連 (4.13)、文字列関連 (4.14)、正規表現関連 (4.15)、日付時間関連 (4.16)、論理関数 (4.17)、行/項目情報関連 (4.18)、NULL 値関連 (4.19)、そして型変換関連 (4.20) の 9 つに分けて解説する。

4.13 日付型と時刻型について

mcal では日付時刻について 2 つの型を用意している。一つは日付型で他方は時刻型である。時刻型は時刻だけでなく日付とセットで表現する。内部的にはグレゴリオ暦に基づいた boost C++ ライブラリの `date_time` ライブラリを利用しており、日付型には `boost::gregorian::date` クラスを、時刻型には `boost::posix_time::ptime` クラスを使っている。詳細は boost.org のドキュメントを参照されたい。

`date` クラスは 32 ビット整数で管理されており、1400 年 1 月 1 日から 9999 年 12 月 31 日の範囲をサポートしている。日付の演算は全てグレゴリオ暦に基づいたものとなっている。不正な日付 (例えば、2013/2/29 や 1399/12/31) が与えられたときは NULL 値が出力される。

一方で `ptime` クラスは、64 ビットで管理されており、ミリ秒まで扱える時刻システムであるが、mcal コマンドにおいてはミリ秒を扱うインターフェースは備えていない。また `ptime` クラスは `date` クラスも内部で参照しており、日付をまたいだ時間計算を可能としている。不正な時刻 (例えば、18:62:11) が与えられたときは NULL 値が出力される。

MCMD は CSV テキストを扱うので、日付/時刻は、データ上は文字列で表現される必要がある。それらの文字列を日付型および時刻型に内部で変換して各種演算を行い、最終結果を再度文字列に戻して出力している。文字列のフォーマットは、日付型は 8 桁固定長文字列 (例えば、"20130911")、時刻型は 14 桁固定長文字列 (例え

表 4.7 論理演算子一覧

内容	書式	例
論理積	$bool_1 \&\& bool_2$	"abc"=="abc" && "xyz"=="abc" (0b0)
論理和	$bool_1 bool_2$	"abc"=="abc" "xyz"=="abc" (0b1)
排他的論理和	$bool_1 \wedge\wedge bool_2$	"abc"=="abc" ^ "xyz"=="abc" (0b1)

例の括弧内は結果を表す (内容は定数の表記で示している)。

表 4.8 論理積

$bool_1$	$bool_2$	結果
1	1	1
1	0	0
0	1	0
0	0	0
null	1	null
null	0	0
null	null	null

表 4.9 論理和

$bool_1$	$bool_2$	結果
1	1	1
1	0	1
0	1	1
0	0	0
null	1	1
null	0	null
null	null	null

表 4.10 排他的論理和

$bool_1$	$bool_2$	結果
1	1	0
1	0	1
0	1	1
0	0	0
null	1	null
null	0	null
null	null	null

表 4.11 演算子の優先順位

優先順位	演算子
1	*, /, %, ^
2	+, -
3	>, <, >=, <=
4	==, !=
5	&&
6	, ^^

ば、"20130911110528")、もしくは 6 桁固定長文字列 (例えば、"110528") を標準としている。

日付型と時刻型と各種関数の関係を図 4.1 に示す。

表 4.12 数値関連関数一覧

節	関数名	機能	出力型
4.95	<code>sum(num₁, num₂, ...)</code>	合計	<i>num</i>
4.22	<code>avg(num₁, num₂, ...)</code>	平均	<i>num</i>
4.94	<code>sqsum(num₁, num₂, ...)</code>	平方和	<i>num</i>
4.65	<code>min(num₁, num₂, ...)</code>	最小値	<i>num</i>
4.63	<code>max(num₁, num₂, ...)</code>	最大値	<i>num</i>
4.75	<code>product(num₁, num₂, ...)</code>	積	<i>num</i>
4.39	<code>factorial(num)</code>	階乗	<i>num</i>
4.45	<code>gcd(num₁, num₂)</code>	最大公約数	<i>num</i>
4.54	<code>lcm(num₁, num₂)</code>	最小公倍数	<i>num</i>
4.93	<code>sqrt(num)</code>	平方根	<i>num</i>
4.14	<code>abs(num)</code>	絶対値	<i>num</i>
4.90	<code>sign(num)</code>	符号	<i>num</i>
4.50	<code>int(num)</code>	整数部	<i>num</i>
4.44	<code>fract(num)</code>	小数部	<i>num</i>
4.88	<code>round(num, 基準値)</code>	四捨五入	<i>num</i>
4.42	<code>floor(num, 基準値)</code>	切り捨て	<i>num</i>
4.26	<code>ceil(num, 基準値)</code>	切り上げ	<i>num</i>
4.74	<code>power(num, 指数)</code>	累乗	<i>num</i>
4.38	<code>exp(num)</code>	指数関数	<i>num</i>
4.60	<code>log(num, 底)</code>	対数	<i>num</i>
4.59	<code>ln(num)</code>	自然対数	<i>num</i>
4.62	<code>log2(num)</code>	底が 2 の対数	<i>num</i>
4.61	<code>log10(num)</code>	常用対数	<i>num</i>
4.34	<code>dist(タイプ, num₁, num₂, ...)</code>	距離	<i>num</i>
4.35	<code>distgps(緯度 1, 経度 1, 緯度 2, 経度 2)</code>	GPS 距離	<i>num</i>
4.47	<code>heron(num₁, num₂, ...)</code>	三角形の面積	<i>num</i>
4.77	<code>rand([乱数の種])</code>	一様乱数	<i>num</i>
4.78	<code>randi(最小値, 最大値 [, 乱数の種])</code>	整数一様乱数	<i>num</i>
4.71	<code>nrnd(平均, 標準偏差 [, 乱数の種])</code>	正規乱数	<i>num</i>
4.73	<code>pi()</code>	円周率	<i>num</i>
4.37	<code>e()</code>	ネイピア数	<i>num</i>
4.43	<code>format()</code>	書式付き出力	<i>str</i>

表 4.13 三角関数関連関数一覧

節	関数名	機能	出力範囲
4.15	<code>acos(num)</code>	コサインの逆関数	$0 \sim \pi$
4.19	<code>asin(num)</code>	サインの逆関数	$-\pi \sim \pi$
4.20	<code>atan(num)</code>	タンジェントの逆関数	$-\pi \sim \pi$
4.21	<code>atan2(num₁, num₂)</code>	座標 (num ₁ , num ₂) の角度	$-\pi \sim \pi$
4.27	<code>cos(r)</code>	コサイン	$-1.0 \sim 1.0$
4.91	<code>sin(r)</code>	サイン	$-1.0 \sim 1.0$
4.96	<code>tan(r)</code>	タンジェント	$-\infty \sim \infty$
4.32	<code>degree(r)</code>	角度	$-\pi \sim \pi$
4.76	<code>radian(角度)</code>	度数を入力したときのラジアンを出力	$-\pi \sim \pi$
4.28	<code>cosh(r)</code>	双曲線余弦	$0 \sim \infty$
4.92	<code>sinh(r)</code>	双曲線正弦	$-\infty \sim \infty$
4.97	<code>tanh(r)</code>	双曲線逆正接	$-1.0 \sim 1.0$

r はラジアンを表した数値。

またユーザは日付/時刻として固定長文字列を標準とせずに、ユリウス通日 (紀元前 4713 年 1 月 1 日正午からの日数) や UNIX 時刻 (1970 年 1 月 1 日 00 時 00 分 00 秒 (GMT) からの経過秒数) などの整数を標準の日時の表記として利用してもよいであろう。ユリウス通日や UNIX 時刻と、日付型/時刻型との変換関数も備えており、十分に運用可能である。ただし、mcal が提供する日付/時刻関数を使う限りにおいては、内部的にはグレゴリオ暦によって管理されて

表 4.14 文字列関連関数一覧

節	関数名	機能	出力型
4.25	<code>cat(token, str₁, str₂, ...)</code>	文字列併合	<i>str</i>
4.57	<code>length(str)</code>	文字列長	<i>num</i>
4.40	<code>fixlen(str, 長さ, 位置, padding 文字)</code>	固定長変換	<i>str</i>
4.87	<code>right(str, 長さ)</code>	末尾切り出し	<i>str</i>
4.56	<code>left(str, 長さ)</code>	先頭切り出し	<i>str</i>
4.64	<code>mid(str, 開始位置, 長さ)</code>	部分文字列切り出し	<i>str</i>
4.102	<code>toupper(str)</code>	小文字大文字変更	<i>str</i>
4.100	<code>tolower(str)</code>	大文字小文字変更	<i>str</i>
4.24	<code>capitalize(str)</code>	先頭文字大文字変換	<i>str</i>
4.51	<code>match(検索文字列, str₁, str₂, ...)</code>	検索	<i>bool</i>
4.46	<code>hasspace(str)</code>	空白類文字検索	<i>bool</i>

表 4.15 正規表現関連関数一覧

節	関数名	機能	出力型
4.80	<code>regextm(str, 正規表現)</code>	全体マッチ	<i>bool</i>
4.84	<code>regexts(str, 正規表現)</code>	マッチ	<i>bool</i>
4.83	<code>regexrep(str, 正規表現, 置換文字列)</code>	マッチ文字列の置換	<i>str</i>
4.79	<code>regexlen(str, 正規表現)</code>	マッチ文字数	<i>num</i>
4.82	<code>regexpos(str, 正規表現)</code>	開始位置	<i>num</i>
4.86	<code>regexstr(str, 正規表現)</code>	マッチ文字列	<i>str</i>
4.81	<code>regexpfx(str, 正規表現)</code>	マッチ文字列のプレフィックス	<i>str</i>
4.85	<code>regextsfx(str, 正規表現)</code>	マッチ文字列のサフィックス	<i>str</i>

表 4.16 日付時間関連関数一覧

節	関数名	機能	出力型
4.99	<code>today()</code>	本日の日付	<i>date</i>
4.69	<code>now()</code>	現在時刻	<i>time</i>
4.103	<code>tseconds(time)</code>	経過秒数	<i>num</i>
4.55	<code>leapyear(dt)</code>	閏年判定	<i>bool</i>
4.106	<code>year(dt)</code>	西暦年	<i>num</i>
4.67	<code>month(dt)</code>	月	<i>num</i>
4.30	<code>day(dt)</code>	日	<i>num</i>
4.105	<code>week(dt)</code>	週番号	<i>num</i>
4.36	<code>dow(dt)</code>	曜日	<i>num</i>
4.98	<code>time(time)</code>	時分秒	<i>str</i>
4.31	<code>date(time)</code>	年月日	<i>str</i>
4.48	<code>hour(time)</code>	時	<i>num</i>
4.66	<code>minute(time)</code>	分	<i>num</i>
4.89	<code>second(time)</code>	秒	<i>num</i>
4.16	<code>age(dt₁, dt₂)</code>	年令	<i>num</i>
4.33	<code>diff(dt₁, dt₂)</code>	期間	<i>num</i>
4.104	<code>uxt(dt)</code>	UNIX 時変換	<i>num</i> (UNIX 時刻)
4.53	<code>julian(dt)</code>	ユリウス通日変換	<i>num</i> (ユリウス通日)

dt は、*date*, *time* の何れかを表す。

おり、その範囲は、1400 年 1 月 1 日から 9999 年 12 月 31 日に限定されることに注意する。また UNIX 時刻は 32 ビット整数で管理されているため、2038 年 1 月 19 日 3 時 14 分 7 秒を超えると正しく計算できないことに注意する。ただユリウス通日や UNIX 時刻を利用する欠点は、その数字を見ただけでは実際にいつの日付時刻なのか理解出来ない点

表 4.17 論理関連関数一覧

節	関数名	機能	出力型
4.17	<code>and(bool₁, bool₂, ...)</code>	論理積	<i>bool</i>
4.72	<code>or(bool₁, bool₂, ...)</code>	論理和	<i>bool</i>
4.68	<code>not(bool)</code>	否定	<i>bool</i>
4.49	<code>if(bool, num₁, num₂)</code>	条件選択	<i>num</i>
4.49	<code>if(bool, str₁, str₂)</code>		<i>str</i>
4.49	<code>if(bool, date₁, date₂)</code>		<i>date</i>
4.49	<code>if(bool, time₁, time₂)</code>		<i>time</i>

表 4.18 行/項目情報関連関数一覧

節	関数名	機能	出力型
4.58	<code>line()</code>	現在処理中の行番号を返す	<i>num</i>
4.101	<code>top()</code>	先頭行	<i>bool</i>
4.23	<code>bottom()</code>	終端行	<i>bool</i>
4.41	<code>fldsize()</code>	項目数	<i>num</i>
4.18	<code>argsize(str₁, str₂, ...)</code>	引数の数	<i>num</i>

表 4.19 NULL 値関連関数一覧

節	関数名	機能	出力型
4.70	<code>nulln()</code>	NULL 値	<i>num</i>
4.70	<code>nulls()</code>		<i>str</i>
4.70	<code>nulld()</code>		<i>date</i>
4.70	<code>nullt()</code>		<i>time</i>
4.70	<code>nullb()</code>		<i>bool</i>
4.52	<code>isnull(num)</code>	NULL 値判定	<i>bool</i>
4.52	<code>isnull(str)</code>		<i>bool</i>
4.52	<code>isnull(date)</code>		<i>bool</i>
4.52	<code>isnull(time)</code>		<i>bool</i>
4.52	<code>isnull(bool)</code>		<i>bool</i>
4.29	<code>countnull(num₁, num₂, ...)</code>	NULL 値の数	<i>num</i>
4.29	<code>countnull(str₁, str₂, ...)</code>		<i>num</i>
4.29	<code>countnull(date₁, date₂, ...)</code>		<i>num</i>
4.29	<code>countnull(time₁, time₂, ...)</code>		<i>num</i>
4.29	<code>countnull(bool₁, bool₂, ...)</code>		<i>num</i>

表 4.20 型変換関連関数一覧

4.107	<i>num</i>	<i>str</i>	<i>date</i>	<i>time</i>	<i>bool</i>
<i>num</i>		<code>n2s(num)</code>			<code>n2b(num)</code>
<i>str</i>	<code>s2n(str)</code>		<code>s2d(str)</code>	<code>s2t(str)</code>	<code>s2b(str)</code>
<i>date</i>		<code>d2s(date)</code>		<code>d2t(date)</code>	
<i>time</i>		<code>t2s(time)</code>	<code>t2d(time)</code>		
<i>bool</i>	<code>b2n(bool)</code>	<code>b2s(bool)</code>			

各セルの関数は、「行ラベル → 列ラベル」の変換関数を示している。

空白セルは、そのような変換関数が定義されていないことを意味する。

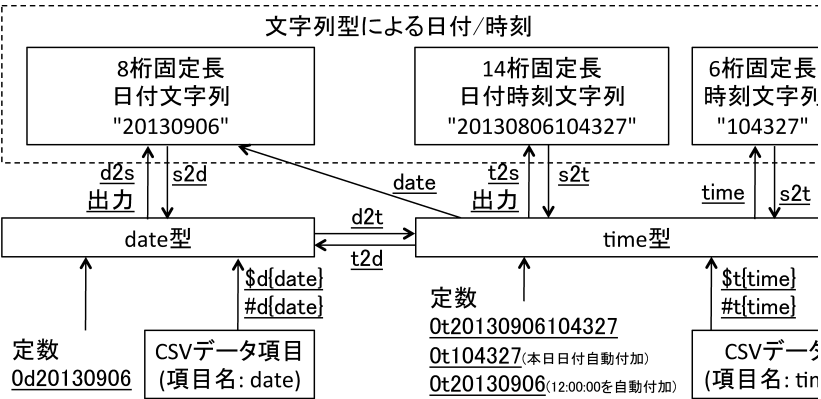


図 4.1 2013 年 9 月 6 日 10 時 43 分 27 秒を例に、date 型と time 型と各種関数の関係を図示している。実線で囲われたボックスは実データを表し、アンダーラインを付したものは関数等を表している。

にあらう。

4.14 abs 絶対値

書式: `abs(num)`

`num` の絶対値を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,1.0
2,-2.5
3,
4,0
$ mcal c='abs(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=abs(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,1.0,1
2,-2.5,2.5
3,,
4,0,0
```

4.15 acos コサインの逆関数

書式: `acos(num)`

アークコサイン (コサインの逆関数) を計算する。パラメータの範囲は $-1.0 \sim 1.0$ で戻り値の範囲は $0.0 \sim \pi$ である。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,-1.0
2,0
3,1.0
4,
5,2
$ mcal c='acos(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=acos(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,-1.0,3.141592654
2,0,1.570796327
3,1.0,0
4,,
5,2,
```

4.16 age 年齢

書式 1: age(生年月日,*date*)

書式 2: age(生年月日,*time*)

日付 *date*(書式 1) もしくは時刻 *time*(書式 2) における年齢を返す。生年月日は書式 1,2 によって日付型/時刻型を指定する。

利用例

例 1: 基本例

2013 年 9 月 1 日における年齢を求める。

```
$ more dat1.csv
id,dob
1,19641010
2,20000101
3,
4,19770812
$ mcal c='age(${dob},0d20130901)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=age(${dob},0d20130901) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,dob,rsl
1,19641010,48
2,20000101,13
3,,
4,19770812,36
```

4.17 and 論理積

書式: `and(bool1, bool2, ...)`

`booli` で与えられた真偽値全ての論理積を計算する。NULL 値を含めた真偽値表は表 4.8 を参照のこと。

利用例

例 1: 基本例

```
$ more dat1.csv
id,b1,b2,b3
1,1,0,1
2,1,1,1
3,1,,1
4,1,1,1
$ mcal c='and(${b1},${b2},${b3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=and(${b1},${b2},${b3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,b1,b2,b3,rsl
1,1,0,1,0
2,1,1,1,1
3,1,,1,
4,1,1,1,1
```

例 2: ワイルドカードを利用した例

`b` から始まる項目 (`b1,b2,b3`) をワイルドカード「`b*`」によって指定している。

```
$ mcal c='and(${b*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=and(${b*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,b1,b2,b3,rsl
1,1,0,1,0
2,1,1,1,1
3,1,,1,
4,1,1,1,1
```

4.18 argsize 引数の数

書式 1: `argsize(str1, str2, ...)`

`stri` で与えられた文字列の数を返す。ワイルドカードをで項目名を指定することで意味が出てくる。すなわちワイルドカード条件を満たす多数の項目の数をカウントすることができる。データ型としては文字列型のみに対応していることに注意する。

利用例

例 1: 基本例

"v"で始まる項目名の数を数える。

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='argsize(${v*})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=argsize(${v*}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,3
2,-5,2,1,3
3,1,,3,3
4,,,3
```

4.19 asin サインの逆関数

書式: `asin(num)`

アークサイン (サインの逆関数) を計算する。パラメータの範囲は $-1.0 \sim 1.0$ で戻り値の範囲は $-\pi/2 \sim \pi/2$ である。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,-1.0
2,0
3,1.0
4,
5,2
$ mcal c='asin(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=asin(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,-1.0,-1.570796327
2,0,0
3,1.0,1.570796327
4,,
5,2,
```

4.20 atan タンジェントの逆関数

書式: `atan(num)`

アークタンジェント (タンジェントの逆関数) を計算する。パラメータの範囲は $-\infty \sim \infty$ で戻り値の範囲は $-\pi/2 \sim \pi/2$ である。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,-1.0
2,0
3,1.0
4,
5,1.0e+10
$ mcal c='atan(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=atan(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,-1.0,-0.7853981634
2,0,0
3,1.0,0.7853981634
4,,
5,1.0e+10,1.570796327
```


4.21 atan2 座標の角度

書式: `atan2(num1, num2)`

x, y 座標 (num_1, num_2) と原点を結ぶ線分と x 軸とが作る角度をラジアンで返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,x,y
1,5,10
2,10,20
3,-1,0
4,0,0
5,,
$ mcal c='atan2(${x},${y})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=atan2(${x},${y}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,x,y,rsl
1,5,10,1.107148718
2,10,20,1.107148718
3,-1,0,3.141592654
4,0,0,0
5,,,
```

4.22 avg 平均

書式: `avg(num1, num2, ...)`

`numi` で与えられた数値の平均を計算する。NULL 値は無視されるが、全てが NULL 値であれば結果も NULL となる。

利用例

例 1: 基本例

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='avg(${v1},${v2},${v3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=avg(${v1},${v2},${v3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,2
2,-5,2,1,-0.6666666667
3,1,,3,2
4,,,,
```

例 2: ワイルドカードを利用した例

v から始まる項目 (v1,v2,v3) をワイルドカード「v*」によって指定している。

```
$ mcal c='avg(${v*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=avg(${v*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,1,2,3,2
2,-5,2,1,-0.6666666667
3,1,,3,2
4,,,,
```

4.23 bottom 終端行

書式: bottom()

終端行であれば真を、そうでなければ偽を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
val
1
2
3
4
$ mcal c='bottom()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=bottom() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
val,rsl
1,0
2,0
3,0
4,1
```

4.24 capitalize 先頭文字大文字変換

書式: `capitalize(str)`

文字列の先頭文字のみ大文字に変更する。アルファベット 26 文字以外の文字については何の影響もない。

利用例

例 1: 基本例

`str` 項目の先頭文字を大文字に変換する。

```
$ more dat1.csv
id,str
1,abc
2,aBd
3,
4,#abc
$ mcal c='capitalize(${str})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc1 a=rsl c=capitalize(${str}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abc,Abc
2,aBd,ABd
3,,
4,#abc,#abc
```

4.25 cat 文字列併合

書式: `cat(token, str1, str2, ...)`

`token` を区切り文字として、指定された `stri` をその順番に併合して一つの文字列を生成する。`token` として空文字 "" を指定すれば、文字列の単純な結合となる。

利用例

例 1: 基本例

3つの項目 `str1, str2, str3` を "#" の区切り文字を挿入して併合する。

```
$ more dat1.csv
id,str1,str2,str3
1,abc,def,ghi
2,A,,CDE
3,,,
4,,XY
$ mcal c='cat("#",$s{str1},$s{str2},$s{str3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=cat("#",$s{str1},$s{str2},$s{str3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str1,str2,str3,rsl
1,abc,def,ghi,abc#def#ghi
2,A,,CDE,A##CDE
3,,,##
4,,XY,##XY
```

例 2: token が空文字の場合

```
$ mcal c='cat("$",$s{str1},$s{str2},$s{str3})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=cat("$",$s{str1},$s{str2},$s{str3}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,str1,str2,str3,rsl
1,abc,def,ghi,abcdefghi
2,A,,CDE,ACDE
3,,,
4,,XY,XY
```

例 3: ワイルドカードを利用した例

`str` から始まる項目 (`str1, str2, str3`) をワイルドカード「`str*`」によって指定している。

```
$ mcal c='cat("$",$s{str*})' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=cat("$",$s{str*}) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,str1,str2,str3,rsl
1,abc,def,ghi,abcdefghi
2,A,,CDE,ACDE
3,,,
4,,XY,XY
```

4.26 ceil 切り上げ

書式: `ceil(num, 基数)`

`num` を切り上げにより丸める。この時、基数の整数倍の値集合のうち、`num` より大きい最小の目盛点にまるめられる。例えば、`ceil(3.42,0.5)` の場合、0.5 とびに目盛がうたれた数直線上で、3.42 より大きい最小の目盛点、すなわち 3.5 が基数 0.5 における切り上げ点となる。基数を省略すると 1.0 がデフォルト値として用いられる。これは小数点以下 1 桁目を切り上げて整数値にまるめることに等しい。

利用例

例 1: 基本例

小数点以下一桁目を切り上げる。

```
$ more dat1.csv
id,val
1,3.28
2,3.82
3,
4,-0.6
$ mcal c='ceil(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=ceil(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.28,4
2,3.82,4
3,,
4,-0.6,-0
```

例 2: 基本例

小数点以下二桁目を切り上げる。

```
$ mcal c='ceil(${val},0.1)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=ceil(${val},0.1) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val,rsl
1,3.28,3.3
2,3.82,3.9
3,,
4,-0.6,-0.5
```

例 3: 基数 0.5 の例

0.5 を基数として切り上げる。

```
$ mcal c='ceil(${val},0.5)' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=ceil(${val},0.5) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,val,rsl
1,3.28,3.5
2,3.82,4
3,,
4,-0.6,-0.5
```

例 4: 基数 10 の例

一桁目を切り上げる。

```
$ more dat2.csv
id,val
1,1341.28
2,188
3,1.235E+3
4,-1.235E+3
$ mcal c='ceil(${val},10)' a=rsl i=dat2.csv o=rsl4.csv
#END# kgc al a=rsl c=ceil(${val},10) i=dat2.csv o=rsl4.csv
$ more rsl4.csv
id,val,rsl
1,1341.28,1350
2,188,190
3,1.235E+3,1240
4,-1.235E+3,-1230
```

4.27 cos コサイン

書式: $\cos(r)$

ラジアン r に対するコサインを計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.141592
2,1.047197
3,
4,6.283185
$ mcal c='cos(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=cos(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.141592,-1
2,1.047197,0.5000004774
3,,
4,6.283185,1
```


4.28 cosh 双曲線余弦

書式: $\cosh(r)$

双曲線余弦 (ハイパーボリック コサイン) を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.141592
2,-1.047197
3,
4,6.283185
$ mcal c='cosh(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=cosh(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.141592,11.59194573
2,-1.047197,1.600286169
3,,
4,6.283185,267.7466792
```

4.29 countnull 合計

書式 1: `countnull(num1, num2, ...)`

書式 2: `countnull(str1, str2, ...)`

書式 3: `countnull(date1, date2, ...)`

書式 4: `countnull(time1, time2, ...)`

書式 5: `countnull(bool1, bool2, ...)`

`numi`(その他の型も同様) で与えられた数値の中で NULL 値の数を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
a,b,c,d
1,,3,4
1,,,
,,,
$ mcal c='countnull(${a},${b},${c},${d})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=countnull(${a},${b},${c},${d}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
a,b,c,d,rsl
1,,3,4,1
1,,,3
,,,4
```

例 2: 他の項目型として項目名をワイルドカードで指定

```
$ mcal c='countnull(${s*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=countnull(${s*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
a,b,c,d,rsl
1,,3,4,1
1,,,3
,,,4
```

4.30 day 日

書式 1: day(*date*)

書式 2: day(*time*)

書式 3: days(*date*)

書式 4: days(*time*)

日付 *date* もしくは時刻 *time* から日を取り出す。書式 1,2 では数値として、書式 3,4 では 2 桁固定長文字列として返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19770812
$ mcal c='day(${date})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=day(${date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,rsl
1,20000101,1
2,20121021,21
3,,
4,19770812,12
```

例 2: 時刻型でも可能

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='day(${time})' a=rsl i=dat2.csv o=rsl2.csv
#END# kgcal a=rsl c=day(${time}) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20000101000000,1
2,20121021111213,21
3,,
4,19770812122212,12
```

4.31 date 年月日

書式: `date(time)`

時刻 *time* から年月日を 8 桁固定長文字列として取り出す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='date(${time})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=date(${time}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,rsl
1,20000101000000,20000101
2,20121021111213,20121021
3,,
4,19770812122212,19770812
```

4.32 degree 角度

書式: degree(r)

ラジアン r に対応する角度を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.141592
2,1.047197
3,
4,6.283185
$ mcal c='degree(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=degree(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.141592,179.9999626
2,1.047197,59.99996842
3,,
4,6.283185,359.9999824
```

4.33 diff 期間

書式 1: `diffyear(date1, date2)`

書式 2: `diffyear(time1, time2)`

書式 3: `diffmonth(date1, date2)`

書式 4: `diffmonth(time1, time2)`

書式 5: `diffday(date1, date2)`

書式 6: `diffday(time1, time2)`

書式 7: `diffhour(date1, date2)`

書式 8: `diffhour(time1, time2)`

書式 9: `diffminute(date1, date2)`

書式 10: `diffminute(time1, time2)`

書式 11: `diffsecond(date1, date2)`

書式 12: `diffsecond(time1, time2)`

`date1(time1)` と `date2(time2)` の差を年数 (月数、日数、時間数、分数、秒数) の単位で計算する。端数は切り捨てられる。

利用例

例 1: 月単位での期間

`date` 項目から 2013 年 9 月 1 日までの経過期間を月数で計算する。

```
$ more dat1.csv
id,date
1,19641010
2,20000101
3,
4,19770812
$ mcal c='diffmonth($d{date},0d20130901)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=diffmonth($d{date},0d20130901) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,rsl
1,19641010,-587
2,20000101,-164
3,,
4,19770812,-433
```

例 2: 分単位での期間

`time` 項目から 2012 年 1 月 1 日 00 時 00 分 00 秒までの経過期間を分単位で計算する。

```
$ more dat2.csv
id,time
1,20120101000000
2,20120101011112
3,
4,20111231235000
$ mcal c='diffmonth($t{time},0t20120101000000)' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=diffmonth($t{time},0t20120101000000) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20120101000000,0
2,20120101011112,0
3,,
4,20111231235000,0
```

4,20111231235000,-1

4.34 dist 距離

書式: `dist(タイプ,num1,num2,...,nk,numk+1,numk+2,...,num2k)`

2つの k 次元ベクトル $(num_1, num_2, \dots, n_k), (num_{k+1}, num_{k+2}, \dots, num_{2k})$ の距離を計算する。距離としては以下のものが利用できる。詳細な定義は `msim` を参照のこと。

- `euclid`: ユークリッド距離
- `cityblock`: 都市ブロック距離
- `hamming`: ハミング距離

ハミング距離については、値は文字型として指定しなければならない(以下の例を参考のこと)。

利用例

例 1: ユークリッド距離

```
$ more dat1.csv
id,x1,y1,x2,y2
1,0,0,1,1
2,0,1,2,0
3,,,,
$ mcal c='dist("euclid",{x1},{y1},{x2},{y2})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=dist("euclid",{x1},{y1},{x2},{y2}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,x1,y1,x2,y2,rsl
1,0,0,1,1,1.414213562
2,0,1,2,0,2.236067977
3,,,,,
```

例 2: 都市ブロック距離

```
$ mcal c='dist("cityblock",{x1},{y1},{x2},{y2})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=dist("cityblock",{x1},{y1},{x2},{y2}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,x1,y1,x2,y2,rsl
1,0,0,1,1,2
2,0,1,2,0,3
3,,,,,
```

例 3: ハミング距離

ハミング距離の計算では、値を文字列として指定していることに注意する。

```
$ more dat2.csv
id,x1,y1,x2,y2
1,a,b,a,c
2,0,1,0,1
3,,,,
$ mcal c='dist("hamming",{s{x1},{s{y1},{s{x2},{s{y2})' a=rsl i=dat2.csv o=rsl3.csv
#END# kgcal a=rsl c=dist("hamming",{s{x1},{s{y1},{s{x2},{s{y2}) i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,x1,y1,x2,y2,rsl
1,a,b,a,c,1
2,0,1,0,1,2
3,,,,,
```


4.35 distgps GPS 距離

書式: distgps(緯度 1, 経度 1, 緯度 2, 経度 2[, 方位])

2 地点の緯度経度座標からその直線距離 (km 単位) を求める。緯度経度は 10 進表記で与える。地球の球状を考慮した計算を行っている。緯度経度の指定は、60 進表記には対応しておらず 10 進表記に変換しておく必要がある。60 進法で表記された度分秒をそれぞれ d, m, s とすると、10 進の座標は $d + m/60 + s/60/60$ で計算できる。

また方位については、北緯と東経はプラスで表記し、南緯と西経はマイナスで表記する。

例えば、大阪駅 (北緯 34.702398, 東経 135.495188), 東京駅間 (北緯 35.681391, 東経 139.766103) の距離を求めるには以下のとおり指定すればよい。

```
distgps(34.702398,135.495188,35.681391,139.766103)
```

またエベレスト (北緯 32.655556, 東経 79.015833), からアコンカグア (南緯 27.987778, 西経 86.944444) の距離は、以下のとおり指定すればよい。

```
distgps(32.655556,79.015833,-27.987778,-86.944444)
```

利用例

例 1: 基本例

```
$ more dat1.csv
point1,point2,lat1,lon1,lat2,lon2
osaka,tenma,34.702398,135.495188,34.704923,135.512233
osaka,tokyo,34.702398,135.495188,35.681391,139.766103
osaka,kobe,34.702398,135.495188,34.679453,135.178221
osaka,Fuji,34.702398,135.495188,35.360556,138.727500
Evelest,Aconcagua,32.655556,79.015833,-27.987778,-86.944444
Denali,Kilimanjaro,63.069444,-151.007222,-3.075833,37.353333
$ mcal c='distgps(${lat1},${lon1},${lat2},${lon2})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=distgps(${lat1},${lon1},${lat2},${lon2}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
point1,point2,lat1,lon1,lat2,lon2,rsl
osaka,tenma,34.702398,135.495188,34.704923,135.512233,1.585046048
osaka,tokyo,34.702398,135.495188,35.681391,139.766103,405.774306
osaka,kobe,34.702398,135.495188,34.679453,135.178221,29.12042213
osaka,Fuji,34.702398,135.495188,35.360556,138.727500,304.7527532
Evelest,Aconcagua,32.655556,79.015833,-27.987778,-86.944444,16956.12242
Denali,Kilimanjaro,63.069444,-151.007222,-3.075833,37.353333,11362.37758
```

4.36 dow 曜日

書式 1: `dow(dt)` 曜日番号 (1~7)

書式 2: `dowj(dt)` 日本語曜日

書式 3: `dowe(dt)` 英語曜日

書式 4: `dowes(dt)` 英語短縮曜日

日付 *date* もしくは時刻 *time* から曜日を返す。曜日の表記によって書式 1~4 を使い分ける。曜日番号は、ISO8601 の規定に従い、1 が月曜日で 7 が日曜日に対応する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19770812
$ mcal c='dow($d{date})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=dow($d{date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,rsl
1,20000101,6
2,20121021,7
3,,
4,19770812,5
```

例 2: 日本語表記

```
$ mcal c='dowj($d{date})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=dowj($d{date}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,date,rsl
1,20000101,土
2,20121021,日
3,,
4,19770812,金
```

例 3: 英語表記

```
$ mcal c='dowe($d{date})' a=rsl i=dat1.csv o=rsl3.csv
#END# kgcal a=rsl c=dowe($d{date}) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,date,rsl
1,20000101,Saturday
2,20121021,Sunday
3,,
4,19770812,Friday
```

例 4: 英語短縮表記

```
$ mcal c='dowes($d{date})' a=rsl i=dat1.csv o=rsl4.csv
#END# kgcal a=rsl c=dowes($d{date}) i=dat1.csv o=rsl4.csv
$ more rsl4.csv
id,date,rsl
```

```
1,20000101,Sat
2,20121021,Sun
3,,
4,19770812,Fri
```

例 5: 時刻型でも可能

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='dow(${time})' a=rsl i=dat2.csv o=rsl5.csv
#END# kgc a=rsl c=dow(${time}) i=dat2.csv o=rsl5.csv
$ more rsl5.csv
id,time,rsl
1,20000101000000,6
2,20121021111213,7
3,,
4,19770812122212,5
```

4.37 e ネイピア数

書式: `e()`

ネイピア数 (e) を出力する。

利用例

例 1: 基本例

```
$ more dat1.csv
id
1
2
$ mcal c='e()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=e() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,2.718281828
2,2.718281828
```

4.38 exp 指数関数

書式: `exp(num)`

power 関数の特殊形で、 e (ネイピア数) を底とする num 乗を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,exponent
1,1
2,-1
3,
4,0
5,0.5
$ mcal c='exp(${exponent})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=exp(${exponent}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,exponent,rsl
1,1,2.718281828
2,-1,0.3678794412
3,,
4,0,1
5,0.5,1.648721271
```

4.39 factorial 階乗

書式: factorial(*num*)

num の階乗を計算する。結果が実数の最大値を超えると、NULL 値が出力される。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,1
2,5
3,
4,10000
$ mcal c='factorial(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=factorial(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,1,1
2,5,120
3,,
4,10000,
```

例 2: 定数を与える例

5 の階乗を計算する。定数を与えているので、全行同じ結果が出力される。

```
$ mcal c='factorial(5)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=factorial(5) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val,rsl
1,1,120
2,5,120
3,,120
4,10000,120
```

4.40 fixlen 固定長変換

書式 1: `fixlen(str, 長さ, 位置, padding 文字)`

書式 2: `fixlenw(str, 長さ, 位置, padding 文字)`

`str` を固定の長さの文字列に変換する。`str` が指定の長さに満たない場合は、右詰もしくは左詰めで指定した文字を埋め込む。左右の選択では、「位置」パラメータに "L" もしくは "R" を指定する。埋め込む文字は「padding 文字」パラメータに任意の文字を指定する。また、`str` の長さが、指定した固定長の長さを超えた場合、右詰の場合は先頭側が、左詰の場合は末尾側の文字列が削られることに注意する。

マルチバイト文字を対象とした固定長変換については `fixlenw` 関数を利用する。

利用例

例 1: 基本例

`str` 項目を 5 文字の固定長文字列に変換する。5 文字に満たない文字列は右詰 ("R") で "#" を埋める。

```
$ more dat1.csv
id,str
1,abc
2,123
3,
4,1234567
$ mcal c='fixlen(${str},5,"R","#')' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=fixlen(${str},5,"R","#") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abc,##abc
2,123,##123
3,,#####
4,1234567,34567
```

例 2: 左詰め例

左詰 ("L") で "#" を埋める。

```
$ mcal c='fixlen(${str},5,"L","#')' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=fixlen(${str},5,"L","#") i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1,abc,abc##
2,123,123##
3,,#####
4,1234567,12345
```

例 3: マルチバイト文字を含む場合

```
$ more dat2.csv
id,str
1, こんにちは
2, 大阪
$ mcal c='fixlenw(${str},4,"R","埋")' a=rsl i=dat2.csv o=rsl3.csv
#END# kgc al a=rsl c=fixlenw(${str},4,"R","埋") i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,str,rsl
1, こんにちは, んにちは
2, 大阪, 埋埋大阪
```

4.41 fldsize 項目数

書式: fldsize()

入力ファイルの項目数を返す。mcmd では全行同じ項目数を想定しているので、この関数の結果は全行同じ値になる。

利用例

例 1: 基本例

```
$ more dat1.csv
a,b,c,d
1,2,3,4
2,3,4,5
3,,
4,x,y,z
$ mcal c='fldsize()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=fldsize() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
a,b,c,d,rsl
1,2,3,4,4
2,3,4,5,4
3,,,4
4,x,y,z,4
```


4.42 floor 切り捨て

書式: floor(*num*, 基数)

num を切り捨てにより丸める。この時、基数の整数倍の値集合のうち *num* より小さい最大の目盛点にまるめられる。例えば、floor(3.82,0.5) の場合、0.5 とびに目盛がうたれた数直線上で、3.82 より小さい最大の目盛点、すなわち 3.5 が基数 0.5 における切り捨て点となる。基数を省略すると 1.0 がデフォルト値として用いられる。これは小数点以下 1 桁目を切り捨てて整数値にまるめることに等しい。

利用例

例 1: 基本例

小数点以下一桁目を切り捨てる。

```
$ more dat1.csv
id,val
1,3.28
2,3.82
3,
4,-0.6
$ mcal c='floor(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=floor(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.28,3
2,3.82,3
3,,
4,-0.6,-1
```

例 2: 基本例

小数点以下二桁目を切り捨てる。

```
$ mcal c='floor(${val},0.1)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=floor(${val},0.1) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val,rsl
1,3.28,3.2
2,3.82,3.8
3,,
4,-0.6,-0.6
```

例 3: 基数 0.5 の例

0.5 を基数として切り捨てる。

```
$ mcal c='floor(${val},0.5)' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=floor(${val},0.5) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,val,rsl
1,3.28,3
2,3.82,3.5
3,,
4,-0.6,-1
```

例 4: 基数 10 の例

一桁目を切り捨てる。

```
$ more dat2.csv
id,val
1,1341.28
2,188
3,1.235E+3
4,-1.235E+3
$ mcal c='floor(${val},10)' a=rsl i=dat2.csv o=rsl4.csv
#END# kgc al a=rsl c=floor(${val},10) i=dat2.csv o=rsl4.csv
$ more rsl4.csv
id,val,rsl
1,1341.28,1340
2,188,180
3,1.235E+3,1230
4,-1.235E+3,-1240
```

4.43 format 書式付き出力

書式: `format(num, 書式付き出力)`

mcal では数値を内部的には浮動小数点として管理している。それを C 言語の `printf` 関数で利用できる出力書式を指定する事で、数値の出力形式を変更する事ができる。利用できる書式は以下の 3 つである。

- `%f`: 小数形式表記
- `%e,%E`: 指数形式表記 (大文字 `%E` で指数記号を大文字表記)
- `%g,%G`: `f,e` の自動判断 (大文字 `%G` で指数記号を大文字表記)

例えば、小数形式表記の `0.00726` は、指数形式表記では `7.260000e-03` となり、`1265` は、`1.265000e+03` となる。

さらに、`%` の直後に数字もしくはプラス記号を指定することで、表示桁数と符号表記を制御できる。数字は「全体の幅・小数点の幅」によって指定する。例えば、`123.456789` は、`%5.2f` の書式によって `123.46` に、`%8.3f` の書式によって `123.457` となる。

プラス符号を常に表記したければ数字の前にプラス記号を付加すればよい。例えば、`123.456789` は、 `%+5.2f` の書式によって `+123.46` となる。

書式の中には上記で説明した記号以外にも、任意の文字列を含めることが可能である。例えば、`250` は、書式 `"合計 %g 円"` によって、`"合計 250 円"` となる。`%` という文字を表示したければ `"%%"` と記すれば良い。`15` は、書式 `"%g%"` によって、`15%` となる。

利用例

例 1: 基本例

`val` を実数として小数点以下 2 桁に変換する。

```
$ more dat1.csv
id,val
1,0.00726
2,123.456789
3,
4,-0.335
$ mcal c='format(${val},"%.3f")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c='format(${val},"%.3f")' i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,0.00726, 0.007
2,123.456789, 123.457
3,,
4,-0.335, -0.335
```

例 2: 指数表現

`val` を指数表現で出力。

```
$ mcal c='format(${val},"%e")' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c='format(${val},"%e")' i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val,rsl
1,0.00726,7.260000e-03
2,123.456789,1.234568e+02
3,,
4,-0.335,-3.350000e-01
```

4.44 fract 小数部

書式: `fract(num)`

`num` の小数部を返す。符号はそのまま出力される。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.14
2,3
3,
4,-12.56789
5,1.2345e+2
6,1.2345e-10
$ mcal c='fract(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=fract(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.14,0.14
2,3,0
3,,
4,-12.56789,-0.56789
5,1.2345e+2,0.45
6,1.2345e-10,1.2345e-10
```

4.45 gcd 最大公約数

書式: `gcd(num1, num2)`

`num1` と `num2` の最大公約数を求める。実数は整数に変換して (切り下げ) 実行される。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val1,val2
1,12,36
2,6,5
3,,
4,12.1,36.2
$ mcal c='gcd(${val1},${val2})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=gcd(${val1},${val2}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val1,val2,rsl
1,12,36,12
2,6,5,1
3,,,
4,12.1,36.2,12
```

例 2: 定数を与える例

`val1` 項目と 36 の最大公約数を求める。

```
$ mcal c='gcd(${val1},36)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=gcd(${val1},36) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val1,val2,rsl
1,12,36,12
2,6,5,6
3,,,
4,12.1,36.2,12
```

4.46 hasspace 空白類文字検索

書式 1: `hasspace(str, 長さ)`

書式 2: `hasspacew(str, 長さ)`

文字列 `str` に空白類文字列が含まれていれば真を、含まれていなければ偽を返す。空白類文字とは、ASCII コードの 0x20 および 0x09～0x0d までの事をいう。それぞれ、半角スペース (0x20)、水平 tab(0x09)、改行 (0x0a)、垂直タブ (0x0b)、改ページ (0x0c)、復帰 (0x0d) である。マルチバイト文字における空白類文字を検索したければ `hasspacew` を使うこと。

利用例

例 1: 基本例

`str` 項目に空白類文字列が含まれていれば真を返す。`id=1` の行は半角スペース文字が含まれ、`id=2` の行は tab 文字が含まれ、そして `id=4` の行は改行文字が含まれているために真となっている。ここで、`id=3` の行は全角スペースのため、検知できていない。

```
$ more dat1.csv
id,str
1,a b
2,ab      c
3,ab   c
4,
5,"aa
bb"
$ mcal c='hasspace(${str})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=hasspace(${str}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,a b,1
2,ab      c,1
3,ab   c,0
4,,0
5,"aa
bb",1
```

例 2: マルチバイト文字

`hasspacew` 関数を使えば全角スペースも正しく検知できる。

```
$ mcal c='hasspacew(${str})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=hasspacew(${str}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1,a b,1
2,ab      c,1
3,ab   c,1
4,,0
5,"aa
bb",1
```

4.47 heron 三角形の面積

書式: `sim(タイプ,num1,num2,...,nk,numk+1,numk+2,...,num2k,num2k+1,num2k+2,...,num3k)`

k 次元空間の3点の座標 $(num_1, num_2, \dots, n_k), (num_{k+1}, num_{k+2}, \dots, num_{2k}), (num_{2k+1}, num_{2k+2}, \dots, num_{3k})$ によって作られる3角形の面積を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,x1,y1,x2,y2,x3,y3
1,0,0,1,0,0,1
2,0,0,0,2,2,0
4,,,,,
3,0,0,1,1,2,2
$ mcal c='heron(${x1},${y1},${x2},${y2},${x3},${y3})' a=rs1 i=dat1.csv o=rs11.csv
#END# kgcal a=rs1 c=heron(${x1},${y1},${x2},${y2},${x3},${y3}) i=dat1.csv o=rs11.csv
$ more rs11.csv
id,x1,y1,x2,y2,x3,y3,rs1
1,0,0,1,0,0,1,0.5
2,0,0,0,2,2,0,2
4,,,,,
3,0,0,1,1,2,2,0
```

4.48 hour 時

書式 1: hour(*time*) 数値

書式 2: hours(*time*) 2 桁固定長文字列

時刻 *time* から時刻を取り出す。返す型によって、書式 1,2 を使い分ければよい。

利用例

例 1: 基本例

```
$ more dat1.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='hour(${time})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=hour(${time}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,rsl
1,20000101000000,0
2,20121021111213,11
3,,
4,19770812122212,12
```

例 2: 文字列とし出力

```
$ mcal c='hours(${time})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=hours(${time}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20000101000000,00
2,20121021111213,11
3,,
4,19770812122212,12
```


4.49 if 条件選択

書式: `if(bool, num1, num2), if(bool, str1, str2), if(bool, date1, date2), if(bool, time1, time2), if(bool, bool1, bool2)`

第1パラメータの値が真であれば第2パラメータを、偽であれば第3パラメータを返す。第1パラメータが NULL 値であれば NULL 値を返す。第2パラメータと第3パラメータは同じ型でなくてはならないことに注意する。

利用例

例 1: 基本例

time 項目が数値として 120000 以下であれば”AM”、そうでなければ”PM”を出力する。

```
$ more dat1.csv
id,time
1,101215
2,210110
3,
4,120001
$ mcal c='if(${time}<=120000,"AM","PM")' a=ampm i=dat1.csv o=rsl1.csv
#END# kgcal a=ampm c=if(${time}<=120000,"AM","PM") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,ampm
1,101215,AM
2,210110,PM
3,,
4,120001,PM
```

例 2: 異なる型を指定した場合

第2パラメータと第3パラメータに異なる型を指定するとエラーとなる。

```
$ mcal c='if(${time}<=120000,"am",1)' a=ampm i=dat1.csv o=rsl2.csv
#ERROR# unknown function or operator: if_BSN (kgcal)
$ more rsl2.csv
```

例 3: 真偽値による条件選択

`$b{項目名}`によって、データ上の値”1”は真、”0”は偽、そしてその他の値は NULL として解釈される。

```
$ more dat2.csv
id,val
1,1
2,0
3,
4,-2
$ mcal c='if($b{val},"true","false")' a=bool i=dat2.csv o=rsl3.csv
#END# kgcal a=bool c=if($b{val},"true","false") i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,val,bool
1,1,true
2,0,false
3,,
4,-2,
```

例 4: 時刻型として比較

```
$ mcal c='if(${time}<=0t120000,"am","pm")' a=ampm i=dat1.csv o=rsl4.csv
#END# kgc al a=ampm c='if(${time}<=0t120000,"am","pm") i=dat1.csv o=rsl4.csv
$ more rsl4.csv
id,time,ampm
1,101215,am
2,210110,pm
3,,
4,120001,pm
```

例 5: if 関数のネスト

```
$ more dat3.csv
id,val
1,10
2,0
3,-5
4,0
$ mcal c='if(${val}>0,"plus",if(${val}<0,"minus","zero"))' a=sign i=dat3.csv o=rsl5.csv
#END# kgc al a=sign c='if(${val}>0,"plus",if(${val}<0,"minus","zero")) i=dat3.csv o=rsl5.csv
$ more rsl5.csv
id,val,sign
1,10,plus
2,0,zero
3,-5,minus
4,0,zero
```

4.50 int 整数部

書式: `int(num)`

`num` の整数部を返す。符号はそのまま出力される。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.14
2,3
3,
4,-12.56789
5,1.2345e+2
6,1.2345e-10
$ mcal c='int(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=int(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.14,3
2,3,3
3,,
4,-12.56789,-12
5,1.2345e+2,123
6,1.2345e-10,0
```

4.51 match 検索

書式 1: match(検索文字列,*str*₁,*str*₂,...)

書式 2: matcha(検索文字列,*str*₁,*str*₂,...)

書式 3: matchs(検索文字列,*str*₁,*str*₂,...)

書式 4: matchas(検索文字列,*str*₁,*str*₂,...)

*str*₁,*str*₂,... から指定した検索文字列を検索し、マッチすれば真をマッチしなければ偽を返す。

OR 検索か AND 検索か、そして完全一致か部分一致かにより、表 4.21 に示すとおり異なる関数名を指定する。

表 4.21 入力データ

	OR 検索	AND 検索
完全一致	match	matcha
部分一致	matchs	matchas

match 関数は、指定した検索文字列が、*str*₁,*str*₂,... のいずれかに完全一致すれば真を返す。matcha 関数は、指定した検索文字列が、*str*₁,*str*₂,... の全てに完全一致すれば真を返す。matchs 関数は、指定した検索文字列が、*str*₁,*str*₂,... のいずれかに部分一致すれば真を返す。matchas 関数は、指定した検索文字列が、*str*₁,*str*₂,... の全てに部分一致すれば真を返す。NULL 値を含めた OR/AND 論理演算の真偽値表は表 4.8 を参照のこと。

利用例

例 1: OR 完全一致

f1,f2,f3 項目のいずれかが 1 であれば真を返す。

```
$ more dat1.csv
id,f1,f2,f3
1,1,1,1
2,1,0,1
3,,
4,1,,1
$ mcal c='match("1",$s{f1},$s{f2},$s{f3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=match("1",$s{f1},$s{f2},$s{f3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,f1,f2,f3,rsl
1,1,1,1,1
2,1,0,1,1
3,,,0
4,1,,1,1
```

例 2: AND 完全一致

f1,f2,f3 項目の全てが文字列"1"であれば真を返す。

```
$ mcal c='matcha("1",$s{f1},$s{f2},$s{f3})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=matcha("1",$s{f1},$s{f2},$s{f3}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,f1,f2,f3,rsl
1,1,1,1,1
2,1,0,1,0
3,,,0
4,1,,1,0
```

例 3: OR 部分一致

s1,s2,s3 項目のいずれかが、文字列 ab を含んでいれば真を返す。

```
$ more dat2.csv
id,s1,s2,s3
1,ab,abx,x
2,abc,xaby,xxab
3,,,
4,#ac,x,x
$ mcal c='matchs("ab",$s{s1},$s{s2},$s{s3})' a=rsl i=dat2.csv o=rsl3.csv
#END# kgc al a=rsl c=matchs("ab",$s{s1},$s{s2},$s{s3}) i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,s1,s2,s3,rsl
1,ab,abx,x,1
2,abc,xaby,xxab,1
3,,,0
4,#ac,x,x,0
```

例 4: AND 部分一致

文字列 ab が s1,s2,s3 項目の全てに、文字列 ab が含まれて入れば真を返す。

```
$ mcal c='matchas("ab",$s{s1},$s{s2},$s{s3})' a=rsl i=dat2.csv o=rsl4.csv
#END# kgc al a=rsl c=matchas("ab",$s{s1},$s{s2},$s{s3}) i=dat2.csv o=rsl4.csv
$ more rsl4.csv
id,s1,s2,s3,rsl
1,ab,abx,x,0
2,abc,xaby,xxab,1
3,,,0
4,#ac,x,x,0
```

例 5: NULL 値の検索

str 項目が NULL 値であれば真を返す。

```
$ mcal c='match(nulls(),$s{s1},$s{s2},$s{s3})' a=rsl i=dat2.csv o=rsl5.csv
#END# kgc al a=rsl c=match(nulls(),$s{s1},$s{s2},$s{s3}) i=dat2.csv o=rsl5.csv
$ more rsl5.csv
id,s1,s2,s3,rsl
1,ab,abx,x,0
2,abc,xaby,xxab,0
3,,,1
4,#ac,x,x,0
```

4.52 isnull NULL 値判定

書式: `isnull(num)`, `isnull(str)`, `isnull(date)`, `isnull(time)`, `isnull(bool)`

`num`(他のデータ型も同様) で与えられた値が NULL 値であるかどうかを判定する。NULL 値であれば 0b1(真) を、そうでなければ 0b0(偽) を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,a
2,
3,b
$ mcal c='isnull(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=isnull(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,a,0
2,,1
3,b,0
```

例 2: 他の項目型も指定可能

```
$ mcal c='isnull(${s{val}})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=isnull(${s{val}}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val,rsl
1,a,0
2,,1
3,b,0
```

例 3: 空文字を定数で与えた場合

```
$ mcal c='isnull("")' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=isnull("") i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,val,rsl
1,a,1
2,,1
3,b,1
```

4.53 julian ユリウス暦変換

書式 1: `julian(date)`

書式 2: `julian(time)`

書式 3: `julian2d(num)`

書式 4: `julian2t(num)`

書式 1,2 では、日付 *date* もしくは時刻 *time* をユリウス通日に変換する。逆に書式 3,4 では、ユリウス通日を日付型もしくは時刻型に変換する。ここで、日付型が与えられたときは、その日の最初の時刻である 00:00:00 として計算される。

利用例

例 1: 基本例

日付型の *date* 項目を `julian` 関数でユリウス通日に変換し、`julian2d` 関数でまたもとに戻す。

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19700101
$ mcal c='julian($d{date})' a=julian i=dat1.csv o=rsl1.csv
#END# kgcal a=julian c=julian($d{date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,julian
1,20000101,2451545
2,20121021,2456222
3,,
4,19700101,2440588
$ mcal c='julian2d({julian})' a=date2 i=rsl1.csv o=rsl2.csv
#END# kgcal a=date2 c=julian2d({julian}) i=rsl1.csv o=rsl2.csv
$ more rsl2.csv
id,date,julian,date2
1,20000101,2451545,20000101
2,20121021,2456222,20121021
3,,,
4,19700101,2440588,19700101
```

例 2: 時刻型も同様

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19700101000100
$ mcal c='julian($t{time})' a=julian i=dat2.csv o=rsl3.csv
#END# kgcal a=julian c=julian($t{time}) i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,time,julian
1,20000101000000,2451545
2,20121021111213,2456222.467
3,,
4,19700101000100,2440588.001
$ mcal c='julian2t({julian})' a=time2 i=rsl3.csv o=rsl4.csv
#END# kgcal a=time2 c=julian2t({julian}) i=rsl3.csv o=rsl4.csv
$ more rsl4.csv
```

id	time	julian	time2
1	20000101000000	2451545	20000101000000
2	20121021111213	2456222.467	20121021000000
3	,	,	,
4	19700101000100	2440588.001	19700101000000

4.54 lcm 最小公倍数

書式: `lcm(num1, num2)`

`num1` と `num2` の最小公倍数を求める。実数は整数に変換して (切り下げ) 実行される。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val1,val2
1,5,3
2,12,4
3,,
4,5.1,3.1
$ mcal c='lcm(${val1},${val2})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcsl a=rsl c=lcm(${val1},${val2}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val1,val2,rsl
1,5,3,15
2,12,4,12
3,,
4,5.1,3.1,15
```

例 2: 定数を与える例

`val1` 項目と 15 の最小公倍数を求める。

```
$ mcal c='lcm(${val1},15)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcsl a=rsl c=lcm(${val1},15) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val1,val2,rsl
1,5,3,15
2,12,4,60
3,,
4,5.1,3.1,15
```

4.55 leapyear 閏年判定

書式 1: leapyear(*date*)

書式 2: leapyear(*time*)

日付 *date* もしくは時刻 *time* が閏年かどうか判定する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19770812
$ mcal c='leapyear(${date})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=leapyear(${date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,rsl
1,20000101,1
2,20121021,1
3,,
4,19770812,0
```

例 2: 時刻型でも判定可能

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='leapyear(${time})' a=rsl i=dat2.csv o=rsl2.csv
#END# kgcal a=rsl c=leapyear(${time}) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20000101000000,1
2,20121021111213,1
3,,
4,19770812122212,0
```

4.56 left 先頭切り出し

書式 1: `left(str, 長さ)`

書式 2: `leftw(str, 長さ)`

文字列 `str` について先頭から長さパラメータで指定した文字数を切り出す。マルチバイト文字を含む場合は `leftw` を使うこと。

利用例

例 1: 基本例

`str` 項目の先頭から 3 文字を切り出す。

```
$ more dat1.csv
id,str
1,abcdefg
2,12345678
3,
4,12
$ mcal c='left(${str},3)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=left(${str},3) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abcdefg,abc
2,12345678,123
3,,
4,12,12
```

例 2: マルチバイト文字を含む例

マルチバイト文字を含む場合は `leftw` を使う。

```
$ more dat2.csv
id,str
1, あいうえお
2,1 あ 2345678
3,1 あ
4, ああ
$ mcal c='leftw(${str},3)' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc a=rsl c=leftw(${str},3) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1, あいうえお, あいう
2,1 あ 2345678,1 あ 2
3,1 あ,1 あ
4, ああ, ああ
```

4.57 length 文字列長

書式 1: `length(str)`

書式 2: `lengthw(str)`

文字列の長さを計算する。`lengthw` 関数を用いると、`str` をワイド文字として扱う。NULL 値の長さは 0 であることに注意する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,str
1,abc
2,3.1415
3,
4,hello world!
$ mcal c='length(${str})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=length(${str}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abc,3
2,3.1415,6
3,,0
4,hello world!,12
```

例 2: マルチバイト文字を含む例

以下は utf-8 でエンコーディングされた日本語を用いた例である。utf-8 の日本語は 1 文字 3 バイトでエンコーディングされているので、`length` 関数では日本語としての文字数ではなく、そのバイト数を返す。

```
$ more dat2.csv
id,str
1, こんにちは
2, 大阪
$ mcal c='length(${str})' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=length(${str}) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1, こんにちは,15
2, 大阪,6
```

例 3: ワイド文字として扱う例

`lengthw` を使うと、内部で文字列をワイド文字に変換するので、マルチバイト文字 1 文字を正しく認識して計算する。

```
$ mcal c='lengthw(${str})' a=rsl i=dat2.csv o=rsl3.csv
#END# kgc al a=rsl c=lengthw(${str}) i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,str,rsl
1, こんにちは,5
2, 大阪,2
```

4.58 line 行番号

書式: line()

mcal コマンドが処理中の行番号を返す。mcmd では行番号は全て 0 から始まるように統一されており、line 関数においても、データの先頭行の行番号は 0 であることに注意する。

利用例

例 1: 基本例

0 から始まる番号が出力される。

```
$ more dat1.csv
id
1
2
3
4
$ mcal c='line()' a=no i=dat1.csv o=rsl1.csv
#END# kgcal a=no c=line() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,no
1,0
2,1
3,2
4,3
```

例 2: 1 から始める

1 から始まる番号を出力する。

```
$ mcal c='line()+1' a=no i=dat1.csv o=rsl2.csv
#END# kgcal a=no c=line()+1 i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,no
1,1
2,2
3,3
4,4
```

4.59 ln 自然対数

書式: $\ln(\text{num}, \text{底})$

num の自然対数を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,10
2,2.718281828
3,
4,0
5,1
6,-8
$ mcal c='ln(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=ln(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,10,2.302585093
2,2.718281828,0.9999999998
3,,
4,0,
5,1,0
6,-8,
```

4.60 log 対数

書式: `log(num, 底)`

`num` についての指定された底の対数を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val,base
1,100,10
2,256,2
3,,
4,2,0
5,0,2
6,1,10
7,-8,2
$ mcal c='log(${val},${base})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=log(${val},${base}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,base,rsl
1,100,10,2
2,256,2,8
3,,
4,2,0,-0
5,0,2,
6,1,10,0
7,-8,2,
```

4.61 log10 常用対数

書式: `log10(num, 底)`

num の常用対数を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,10
2,0.1
3,
4,0
5,1
6,-8
$ mcal c='log10(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=log10(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,10,1
2,0.1,-1
3,,
4,0,
5,1,0
6,-8,
```


4.62 log2 底が 2 の対数

書式: `log2(num)`

`num` について 2 を底とする対数を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,10
2,256
3,
4,0
5,1
6,-8
$ mcal c='log2(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=log2(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,10,3.321928095
2,256,8
3,,
4,0,
5,1,0
6,-8,
```

4.63 max 最大値

書式: $\max(num_1, num_2, \dots)$

num_i で与えられた数値の最大値を計算する。NULL 値は無視されるが、全てが NULL 値であれば結果も NULL となる。

利用例

例 1: 基本例

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='max(${v1},${v2},${v3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc1 a=rsl c=max(${v1},${v2},${v3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,3
2,-5,2,1,2
3,1,,3,3
4,,,,
```

例 2: ワイルドカードを利用した例

v から始まる項目 (v1,v2,v3) をワイルドカード「v*」によって指定している。

```
$ mcal c='max(${v*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc1 a=rsl c=max(${v*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,1,2,3,3
2,-5,2,1,2
3,1,,3,3
4,,,,
```

4.64 mid 部分文字列切り出し

書式 1: `mid(str, 開始位置, 長さ)`

書式 2: `midw(str, 開始位置, 長さ)`

文字列 `str` の指定した開始位置から長さ分を切り出す。開始位置は 0 から始まることに注意する。マルチバイト文字を含む場合は `midw` を使うこと。

利用例

例 1: 基本例

`str` 項目の 2 文字目からから 3 文字を切り出す。

```
$ more dat1.csv
id,str
1,abcdefg
2,12345678
3,
4,12
$ mcal c='mid(${str},2,3)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=mid(${str},2,3) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abcdefg,cde
2,12345678,345
3,,
4,12,
```

例 2: 基本例

マルチバイト文字を含む場合は `midw` を使う。

```
$ more dat2.csv
id,str
1, あいうえお
2,1234567 あ 8
3,1 あ
4, ああ
$ mcal c='midw(${str},2,3)' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=midw(${str},2,3) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1, あいうえお, うえお
2,1234567 あ 8,345
3,1 あ,
4, ああ,
```

4.65 min 最小値

書式: $\text{min}(\text{num}_1, \text{num}_2, \dots)$

num_i で与えられた数値の最小値を計算する。NULL 値は無視されるが、全てが NULL 値であれば結果も null となる。

利用例

例 1: 基本例

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='min(${v1},${v2},${v3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=min(${v1},${v2},${v3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,1
2,-5,2,1,-5
3,1,,3,1
4,,,,
```

例 2: ワイルドカードを利用した例

v から始まる項目 (v1,v2,v3) をワイルドカード「v*」によって指定している。

```
$ mcal c='min(${v*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=min(${v*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,1,2,3,1
2,-5,2,1,-5
3,1,,3,1
4,,,,
```

4.66 minute 分

書式 1: `minute(time)` 数値

書式 2: `minutes(time)` 2 桁固定長文字列

時刻 *time* から分を取り出す。返す型によって、書式 1,2 を使い分ければよい。

利用例

例 1: 基本例

```
$ more dat1.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='minute(${time})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=minute(${time}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,rsl
1,20000101000000,0
2,20121021111213,12
3,,
4,19770812122212,22
```

例 2: 文字列とし出力

```
$ mcal c='minutes(${time})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=minutes(${time}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20000101000000,00
2,20121021111213,12
3,,
4,19770812122212,22
```

4.67 month 月

書式 1: month(*dt*) 数値型月番号

書式 2: months(*dt*) 文字列型 2 桁固定長月番号

書式 3: monthe(*dt*) 英語表記

書式 4: monthes(*dt*) 英語短縮表記

日付 *date* もしくは時刻 *time* から月を返す。月の表記によって書式 1~4 を使い分ける。

利用例

例 1: 基本例

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19770812
$ mcal c='month(${date})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=month(${date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,rsl
1,20000101,1
2,20121021,10
3,,
4,19770812,8
```

例 2: 固定長文字列として

```
$ mcal c='months(${date})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=months(${date}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,date,rsl
1,20000101,01
2,20121021,10
3,,
4,19770812,08
```

例 3: 英語表記

```
$ mcal c='monthe(${date})' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=monthe(${date}) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,date,rsl
1,20000101,January
2,20121021,October
3,,
4,19770812,August
```

例 4: 英語短縮表記

```
$ mcal c='monthes(${date})' a=rsl i=dat1.csv o=rsl4.csv
#END# kgc al a=rsl c=monthes(${date}) i=dat1.csv o=rsl4.csv
$ more rsl4.csv
id,date,rsl
1,20000101,Jan
2,20121021,Oct
```

```
3,,  
4,19770812,Aug
```

例 5: 時刻型でも可能

```
$ more dat2.csv  
id,time  
1,20000101000000  
2,20121021111213  
3,  
4,19770812122212  
$ mcal c='month(${time})' a=rsl i=dat2.csv o=rsl5.csv  
#END# kgcsl a=rsl c=month(${time}) i=dat2.csv o=rsl5.csv  
$ more rsl5.csv  
id,time,rsl  
1,20000101000000,1  
2,20121021111213,10  
3,,  
4,19770812122212,8
```

4.68 not 否定

書式: `not(bool)`

`bool` で与えられた真偽値の否定を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,b
1,1
2,
3,0
$ mcal c='not(${b})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=not(${b}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,b,rsl
1,1,0
2,,
3,0,1
```


4.69 now 現在時刻

書式: now()

now 関数は現在時刻を時刻型で返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id
1
2
$ mcal c='now()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=now() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,20160630145136
2,20160630145136
```

例 2: 時刻のみ切り出し

```
$ mcal c='time(now())' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=time(now()) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,rsl
1,145136
2,145136
```

例 3: 日付のみ切り出し

```
$ mcal c='date(now())' a=rsl i=dat1.csv o=rsl3.csv
#END# kgcal a=rsl c=date(now()) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,rsl
1,20160630
2,20160630
```

4.70 null NULL 値

書式: nulln(),nulls(),nulld(),nullt(),nullb()

型に応じた NULL 値を返す。if 関数と組み合わせて NULL 値を出力したい時に使うことができる。

利用例

例 1: 基本例

rsl という項目の全行に NULL 値を出力する。

```
$ more dat1.csv
id
1
2
3
$ mcal c='nulls()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=nulls() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,
2,
3,
```

例 2: if 文の中での利用

if 文の第二パラメータで数値を指定しているので、それに合わせて nulln() 関数を用いる。

```
$ mcal c='if(${id}==1,1,nulln())' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=if(${id}==1,1,nulln()) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,rsl
1,1
2,
3,
```

例 3: isnull と同等の指定

```
$ mcal c='if(${val}==nulln(),"null","notNull")' a=rsl i=dat2.csv o=rsl3.csv
#END# kgc al a=rsl c=if(${val}==nulln(),"null","notNull") i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,val,rsl
1,a,
2,,
3,b,
```

4.71 nrand 正規乱数

指定された平均値と標準偏差による正規乱数を発生させる。

同じ乱数の種を指定すれば、同じ乱数系列となる。指定可能な乱数の種の範囲は-2147483648～2147483647 である。乱数の種を省略すると、時刻 (1/1000 秒単位) に応じた異なる乱数の種が利用される。

乱数の生成にはメルセンヌ・ツイスター法を利用している ([原作者のページ](#), [boost ライブラリ](#))。

利用例

例 1: 基本例

平均 0 標準偏差 1(標準正規分布) に基づく乱数を成する。また、乱数の種は 10 に設定している。

```
$ more dat1.csv
id
1
2
3
4
$ mcal c='nrand(0,1,10)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=nrand(0,1,10) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,0.1125259159
2,-0.8349559194
3,1.158317265
4,0.1518928088
```

4.72 or 論理和

書式: `or(bool1, bool2, ...)`

`booli` で与えられた真偽値全ての論理和を計算する。NULL 値を含めた真偽値表は表 4.9 を参照のこと。

利用例

例 1: 基本例

```
$ more dat1.csv
id,b1,b2,b3
1,1,0,0
2,1,,1
3,0,,0
4,0,0,0
$ mcal c='or($b{b1},$b{b2},$b{b3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=or($b{b1},$b{b2},$b{b3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,b1,b2,b3,rsl
1,1,0,0,1
2,1,,1,1
3,0,,0,
4,0,0,0,0
```

例 2: ワイルドカードを利用した例

`b` から始まる項目 (`b1,b2,b3`) をワイルドカード「`b*`」によって指定している。

```
$ mcal c='or($b{b*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=or($b{b*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,b1,b2,b3,rsl
1,1,0,0,1
2,1,,1,1
3,0,,0,
4,0,0,0,0
```

4.73 pi 円周率

書式: pi()

円周率 (π) を出力する。

利用例

例 1: 基本例

```
$ more dat1.csv
id
1
2
$ mcal c='pi()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=pi() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,3.141592654
2,3.141592654
```

4.74 power 累乗

書式: `power(num, 指数)`

`num` の累乗を計算する。演算子`^`と等価である。結果が実数の最大値を超えると、NULL 値が出力される。

利用例

例 1: 基本例

```
$ more dat1.csv
id,base,exponent
1,5,2
2,2,8
3,,
4,0,10
5,10,0
6,2,0.5
7,2,-1
$ mcal c='power(${base},${exponent})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=power(${base},${exponent}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,base,exponent,rsl
1,5,2,25
2,2,8,256
3,,,
4,0,10,0
5,10,0,1
6,2,0.5,1.414213562
7,2,-1,0.5
```

4.75 product 積

書式: `product(num1, num2, ...)`

`numi` で与えられた数値の積を計算する。NULL 値は無視されるが、全てが NULL 値であれば結果も NULL となる。

利用例

例 1: 基本例

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='product(${v1},${v2},${v3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc1 a=rsl c=product(${v1},${v2},${v3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,6
2,-5,2,1,-10
3,1,,3,3
4,,,,
```

例 2: ワイルドカードを利用した例

v から始まる項目 (v1,v2,v3) をワイルドカード「v*」によって指定している。

```
$ mcal c='product(${v*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc1 a=rsl c=product(${v*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,1,2,3,6
2,-5,2,1,-10
3,1,,3,3
4,,,,
```

4.76 radian ラジアン

角度をラジアンに変換する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,x,y
1,5,10
2,10,20
3,-1,0
4,0,0
5,,
$ mcal c='atan2({x},{y})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=atan2({x},{y}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,x,y,rsl
1,5,10,1.107148718
2,10,20,1.107148718
3,-1,0,3.141592654
4,0,0,0
5,,,
```


4.77 rand 一様乱数

書式: ([乱数の種])

0.0～1.0 の範囲で一様乱数を生成する。単独で利用するとその結果は `rand` コマンドを `-int` オプションなしで実行した結果に等しい。

同じ乱数の種を指定すれば、同じ乱数系列となる。指定可能な乱数の種の範囲は-2147483648～2147483647 である。乱数の種を省略すると、時刻 (ミリ (1/1000 秒単位)) に応じた異なる乱数の種が利用される。

乱数の生成にはメルセンヌ・ツイスター法を利用している ([原作者のページ](#), [boost ライブラリ](#))。

利用例

例 1: 基本例

0.0 から 1.0 の一様乱数を生成する。乱数の種を指定しているので、何度実行しても同じ乱数系列が生成される。

```
$ more dat1.csv
id
1
2
3
4
$ mcal c='rand(1)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=rand(1) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,0.4170219984
2,0.9971848081
3,0.7203244893
4,0.9325573612
```

例 2: 乱数の種未指定

乱数の種を指定していないので、実行の度に異なる乱数系列が生成される。

```
$ mcal c='rand()' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=rand() i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,rsl
1,0.4091324708
2,0.725003521
3,0.3267871595
4,0.7816882059
```

4.78 randi 整数一様乱数

書式: (最小値, 最大値 [, 乱数の種])

最小値～最大値の範囲で整数乱数を生成する。単独で利用するとその結果は `mrnd` コマンドを `-int` オプションを指定して実行した結果に等しい。

同じ乱数の種を指定すれば、同じ乱数系列となる。指定可能な乱数の種の範囲は-2147483648～2147483647 である。乱数の種を省略すると、時刻 (1/1000 秒単位) に応じた異なる乱数の種が利用される。

乱数の生成にはメルセンヌ・ツイスター法を利用している ([原作者のページ](#), [boost ライブラリ](#))。

利用例

例 1: 基本例

100 から 999(3 桁整数 900 種類) の整数乱数を生成する。乱数の種を指定しているので、何度実行しても同じ乱数系列が生成される。

```
$ more dat1.csv
id
1
2
3
4
$ mcal c='randi(100,999,1)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=randi(100,999,1) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,475
2,997
3,748
4,939
```

例 2: 0,1 の整数乱数

0 と 1 の 2 種類の整数乱数を生成する。乱数の種を指定していないので、実行の度に異なる乱数系列が生成される。

```
$ mcal c='randi(0,1)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=randi(0,1) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,rsl
1,0
2,0
3,0
4,1
```

4.79 regexlen マッチ文字数

書式 1: `regexlen(str, 正規表現)`

書式 2: `regexlenw(str, 正規表現)`

指定した正規表現が最長マッチする文字列 *str* の部分文字列の長さを返す。マッチしなければ 0 を返す。すなわち 0 文字にマッチしたと解釈する。*str* もしくは正規表現にマルチバイト文字を含む場合は `regexlenw` 関数を使うこと。

利用例

例 1: 基本例

正規表現 `c.*a` に最も長くマッチする部分文字列の長さを得る。マッチした部分文字列については `regexstr` と同じ入力データを使っているの、比較すると分かりやすい。

```
$ more dat1.csv
id,str
1,xcbbbayy
2,xxcbaay
3,
4,bacabbca
$ mcal c='regexlen(${str},"c.*a")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=regexlen(${str},"c.*a") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,xcbbbayy,5
2,xxcbaay,4
3,,0
4,bacabbca,6
```

例 2: マルチバイト文字

正規表現 `"あ.*い"` に最も長くマッチする部分文字列の長さを得る。ただし、以下ではマルチバイト文字対応でない `regexlen` 関数を使っているの、文字数ではなくバイト数を返している。

```
$ more dat2.csv
id,str
1, 漢漢あ bbb い yy
2, 漢あ b いい y
3,
4,b あいあ bb いあ
$ mcal c='regexlen(${str},"あ.*い")' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=regexlen(${str},"あ.*い") i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1, 漢漢あ bbb い yy,9
2, 漢あ b いい y,10
3,,0
4,b あいあ bb いあ,14
```

例 3: マルチバイト文字 2

正規表現 `"あ.*い"` に最も長くマッチする部分文字列の長さを得る。`regexlenw` 関数を使うと、正しくマルチバイト文字を扱ってくれる。

```
$ mcal c='regexlenw(${str},"あ.*い")' a=rsl i=dat2.csv o=rsl3.csv
#END# kgc al a=rsl c=regexlenw(${str},"あ.*い") i=dat2.csv o=rsl3.csv
$ more rsl3.csv
```

<pre>id,str,rsl 1, 漢漢あ bbb い yy,5 2, 漢あ b いい y,4 3,,0 4,b あいあ bb いあ,6</pre>

4.80 regexm 全体マッチ

書式 1: `regexm(str, 正規表現)`

書式 2: `regexmw(str, 正規表現)`

指定した正規表現が、文字列 *str* 全体にマッチすれば真を返す。*str* もしくは正規表現にマルチバイト文字を含み、Shift_JIS など文字の出現順によっては意に沿わない検索結果となる場合は `regexmw` 関数を使うこと。

利用例

例 1: 基本例

`id=1, id=2` 共に、正規表現で示された *c* に続く *aa* を含んでいるが、`id=2` ではマッチする範囲が部分的 (2 文字目の *c* から最後まで) であるために偽となっている。

```
$ more dat1.csv
id,str
1,caabaa
2,acabaaa
3,
4,bbcbcc
$ mcal c='regexm(${str},"c.*aa")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=regexm(${str},"c.*aa") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,caabaa,1
2,acabaaa,0
3,,0
4,bbcbcc,0
```

例 2: 末尾一致

正規表現 `.*` で *str* 項目の全体がカバーされるのは `id=4` の行のみである。

```
$ mcal c='regexm(${str},".*c")' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=regexm(${str},".*c") i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1,caabaa,0
2,acabaaa,0
3,,0
4,bbcbcc,1
```

例 3: 空文字マッチ

正規表現 `^$` で `id=3` の空文字にマッチする。

```
$ mcal c='regexm(${str},"^$")' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=regexm(${str},"^$") i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,str,rsl
1,caabaa,0
2,acabaaa,0
3,,1
4,bbcbcc,0
```

4.81 regexpfx マッチ文字列のプレフィックス

書式 1: `regexpfx(str, 正規表現)`

書式 2: `regexpfxw(str, 正規表現)`

指定した正規表現が最長マッチする文字列 *str* の部分文字列のプレフィックス (部分文字列より先頭側の文字列) を返す。同じ文字列及び正規表現によって `regexpfx` 関数, `regexstr` 関数, `regxsfx` 関数の 3 関数を実行した場合、それぞれで得られた文字列をその順番に結合すると元の文字列が再現できる関係にある。*str* もしくは正規表現にマルチバイト文字を含み、Shift_JIS など文字の出現順によっては意に沿わない検索結果となる場合は `regexpfxw` 関数を使うこと。

利用例

例 1: 基本例

正規表現 `c.*a` に最も長くマッチする部分文字列のプレフィックスを得る。例えば `id=4` では、*str* 項目の `cabbca` にマッチしており、そのプレフィックスすなわち `ba` を返している。`regexstr`, `regexpfx` と同じ入力データを使っているので、比較すると分かりやすい。

```
$ more dat1.csv
id,str
1,xcbbbayy
2,xxcbaay
3,
4,bacabbca
$ mcal c='regexpfx(${str},"c.*a")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=regexpfx(${str},"c.*a") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,xcbbbayy,x
2,xxcbaay,xx
3,,
4,bacabbca,ba
```

4.82 regexpos マッチ位置

書式 1: regexpos(*str*, 正規表現)

書式 2: regexposw(*str*, 正規表現)

指定した正規表現が最長マッチする文字列 *str* の部分文字列の開始位置を返す。文字列の先頭位置は 0 であることに注意する。またマッチしなければ NULL 値を返す。*str* もしくは正規表現にマルチバイト文字を含む場合は regexposw 関数を使うこと。

利用例

例 1: 基本例

正規表現 `c.*a` に最も長くマッチする部分文字列の位置を得る。先頭文字の位置は 0 であることに注意する。マッチした部分文字列については `regexstr` と同じ入力データを使っているの、比較すると分かりやすい。

```
$ more dat1.csv
id,str
1,xcbbbayy
2,xxcbaay
3,
4,bacabbca
$ mcal c='regexpos(${str},"c.*a")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=regexpos(${str},"c.*a") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,xcbbbayy,1
2,xxcbaay,2
3,,
4,bacabbca,2
```

例 2: マルチバイト文字

正規表現 `"い.*あ"` に最も長くマッチする部分文字列の長さを得る。ただし、以下ではマルチバイト文字対応でない `regexpos` 関数を使っているの、文字数ではなくバイト数でカウントした場合の位置を返している。

```
$ more dat2.csv
id,str
1, 漢漢あ bbb い yy
2, 漢あ b いい y
3,
4,b あいあ bb いあ
$ mcal c='regexpos(${str},"あ.*い")' a=rsl i=dat2.csv o=rsl2.csv
#END# kgcal a=rsl c=regexpos(${str},"あ.*い") i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1, 漢漢あ bbb い yy,6
2, 漢あ b いい y,3
3,,
4,b あいあ bb いあ,1
```

例 3: マルチバイト文字 2

正規表現 `"い.*あ"` に最も長くマッチする部分文字列の長さを得る。regexposw 関数を使うと、正しくマルチバイト文字を扱ってくれる。

```
$ mcal c='regexposw(${str},"あ.*い")' a=rsl i=dat2.csv o=rsl3.csv
#END# kgc al a=rsl c=regexposw(${str},"あ.*い") i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,str,rsl
1, 漢漢あ bbb い yy,2
2, 漢あ b いい y,1
3,,
4,b あいあ bb いあ,1
```


4.83 regexrep マッチ文字列の置換

書式 1: `regexrep(str, 正規表現, 置換文字列)`

書式 2: `regexrepw(str, 正規表現, 置換文字列)`

指定した正規表現が最長マッチした文字列 *str* の部分文字列を置換文字列で置換する。*str* もしくは正規表現にマルチバイト文字を含み Shift_JIS など文字の出現順によっては意に沿わない検索結果となる場合は `regexrepw` 関数を使うこと。

利用例

例 1: 基本例

`id=1,id=2` の *str* 項目にマッチした部分文字列を `MMM` に置換する。

```
$ more dat1.csv
id,str
1,caabaa
2,acabaaa
3,
4,cbcbcc
$ mcal c='regexrep(${str},"c.*aa","MMM")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=regexrep(${str},"c.*aa","MMM") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,caabaa,MMM
2,acabaaa,aMMM
3,,
4,cbcbcc,cbcbcc
```

4.84 regexs マッチ

書式 1: `regexs(str, 正規表現)`

書式 2: `regexsw(str, 正規表現)`

指定した正規表現が、文字列 *str* の一部にでもマッチすれば真を返す。*str* もしくは正規表現にマルチバイト文字を含み、Shift_JIS など文字の出現順によっては意に沿わない検索結果となる場合は `regexsw` 関数を使うこと。

利用例

例 1: 基本例

`id=1, id=2` 共に、正規表現で示された `c` に続く `aa` を含んでいるので真を返す。

```
$ more dat1.csv
id,str
1,caabaa
2,acabaaa
3,
4,cbcbcc
$ mcal c='regexs(${str},"c.*aa")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=regexs(${str},"c.*aa") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,caabaa,1
2,acabaaa,1
3,,0
4,cbcbcc,0
```

例 2: 先頭一致

正規表現 `. *c` を *str* 項目が含むのは `id=3` 以外全ての行である。

```
$ mcal c='regexs(${str},".*c")' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=regexs(${str},".*c") i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1,caabaa,1
2,acabaaa,1
3,,0
4,cbcbcc,1
```

4.85 regexsfx マッチ文字列のサフィックス

書式 1: regexsfx(*str*, 正規表現)

書式 2: regexsfxw(*str*, 正規表現)

指定した正規表現が最長マッチする文字列 *str* の部分文字列のサフィックス (部分文字列より末尾側の文字列) を返す。同じ文字列及び正規表現によって regexpfx 関数, regexstr 関数, regexsfx 関数の 3 関数を実行した場合、それぞれで得られた文字列をその順番に結合すると元の文字列が再現できる関係にある。*str* もしくは正規表現にマルチバイト文字を含み、Shift_JIS など文字の出現順によっては意に沿わない検索結果となる場合は regexsfxw 関数を使うこと。

利用例

例 1: 基本例

正規表現 `c.*a` に最も長くマッチする部分文字列のサフィックスを得る。例えば `id=4` では、*str* 項目の `cabbca` にマッチしており、そのサフィックスすなわち `null` 文字を返している。`regexstr`, `regexpfx` と同じ入力データを使っているので、比較すると分かりやすい。

```
$ more dat1.csv
id,str
1,xcbbbayy
2,xxcbaay
3,
4,bacabbca
$ mcal c='regexsfx(${str},"c.*a")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=regexsfx(${str},"c.*a") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,xcbbbayy,yy
2,xxcbaay,y
3,,
4,bacabbca,
```

4.86 regexstr マッチ文字列

書式 1: `regexstr(str, 正規表現)`

書式 2: `regexstrw(str, 正規表現)`

指定した正規表現が最長マッチする文字列 *str* の部分文字列を返す。*str* もしくは正規表現にマルチバイト文字を含み、Shift_JIS など文字の出現順によっては意に沿わない検索結果となる場合は `regexstrw` 関数を使うこと。

利用例

例 1: 基本例

正規表現 `c.*a` に最も長くマッチする部分文字列を得る。`id=2` では、`cba` もしくは `cbaa` いずれの部分文字列にもマッチしたと考えることができるが、本関数では、より長くマッチした文字列を返す。

```
$ more dat1.csv
id,str
1,xcbbbayy
2,xxcbaay
3,
4,bacabbca
$ mcal c='regexstr(${str},"c.*a")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=regexstr(${str},"c.*a") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,xcbbbayy,cbba
2,xxcbaay,cbaa
3,,
4,bacabbca,cabbca
```

4.87 right 末尾切り出し

書式 1: `right(str, 長さ)`

書式 2: `rightw(str, 長さ)`

文字列 `str` について末尾から長さパラメータで指定した文字数を切り出す。マルチバイト文字を含む場合は `rightw` を使うこと。

利用例

例 1: 基本例

`str` 項目の末尾から 3 文字を切り出す。

```
$ more dat1.csv
id,str
1,abcdefg
2,12345678
3,
4,12
$ mcal c='right(${str},3)' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc a=rsl c=right(${str},3) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abcdefg,efg
2,12345678,678
3,,
4,12,12
```

例 2: マルチバイト文字を含む例

マルチバイト文字を含む場合は `rightw` を使う。

```
$ more dat2.csv
id,str
1, あいうえお
2,1234567 あ 8
3,1 あ
4, ああ
$ mcal c='rightw(${str},3)' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc a=rsl c=rightw(${str},3) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,str,rsl
1, あいうえお, うえお
2,1234567 あ 8,7 あ 8
3,1 あ,1 あ
4, ああ, ああ
```

4.88 round 四捨五入

書式: round(*num*, 基数)

num を四捨五入によりまるめる。この時、基数の整数倍の値集合のうち最も近い値にまるめられる。例えば、round(3.82,0.5) の場合、0.5 とびに目盛がうたれた数直線上で、3.82 に最も近い目盛点、すなわち 4.0 が基数 0.5 における四捨五入点となる。基数を省略すると 1.0 がデフォルト値として用いられる。これは小数点以下 1 桁目を四捨五入して整数値にまるめることに等しい。

利用例

例 1: 基本例

小数点以下一桁目を四捨五入する。

```
$ more dat1.csv
id,val
1,3.28
2,3.82
3,
4,-0.6
$ mcal c='round(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=round(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.28,3
2,3.82,4
3,,
4,-0.6,-1
```

例 2: 基本例

小数点以下二桁目を四捨五入する。

```
$ mcal c='round(${val},0.1)' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=round(${val},0.1) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,val,rsl
1,3.28,3.3
2,3.82,3.8
3,,
4,-0.6,-0.6
```

例 3: 基数 0.5 の例

0.5 を基数として四捨五入する。

```
$ mcal c='round(${val},0.5)' a=rsl i=dat1.csv o=rsl3.csv
#END# kgc al a=rsl c=round(${val},0.5) i=dat1.csv o=rsl3.csv
$ more rsl3.csv
id,val,rsl
1,3.28,3.5
2,3.82,4
3,,
4,-0.6,-0.5
```

例 4: 基数 10 の例

一桁目を四捨五入する。

```
$ more dat2.csv
id,val
1,1341.28
2,188
3,1.235E+3
4,-1.235E+3
$ mcal c='round(${val},10)' a=rsl i=dat2.csv o=rsl4.csv
#END# kgc al a=rsl c=round(${val},10) i=dat2.csv o=rsl4.csv
$ more rsl4.csv
id,val,rsl
1,1341.28,1340
2,188,190
3,1.235E+3,1240
4,-1.235E+3,-1230
```

4.89 second 秒

書式 1: `second(time)` 数値

書式 2: `seconds(time)` 2 桁固定長文字列

時刻 *time* から秒を取り出す。返す型によって、書式 1,2 を使い分ければよい。

利用例

例 1: 基本例

```
$ more dat1.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='second(${time})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=second(${time}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,rsl
1,20000101000000,0
2,20121021111213,13
3,,
4,19770812122212,12
```

例 2: 文字列とし出力

```
$ mcal c='seconds(${time})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=seconds(${time}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20000101000000,00
2,20121021111213,13
3,,
4,19770812122212,12
```


4.90 sign 符号

書式: `sign(num)`

`num` の符号を判定する。プラスであれば 1 を、マイナスであれば -1 を、ゼロであれば 0 を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,5
2,-5
3,
4,0
$ mcal c='sign(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=sign(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,5,1
2,-5,-1
3,,
4,0,0
```

4.91 sin サイン

書式: $\sin(r)$

ラジアン r に対するサインを計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.141592
2,0.523599
3,
4,6.283185
$ mcal c='sin(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=sin(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.141592,6.535897931e-07
2,0.523599,0.5000001943
3,,
4,6.283185,-3.071795869e-07
```

4.92 sinh 双曲線正弦

書式: $\sinh(r)$

双曲線正弦 (ハイパーボリック サイン) を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.141592
2,-1.047197
3,
4,6.283185
$ mcal c='sinh(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=sinh(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.141592,11.54873178
2,-1.047197,-1.249366168
3,,
4,6.283185,267.7448118
```

4.93 sqrt 平方根

書式: `sqrt(num)`

`num` の平方根を求める。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,9
2,2
3,
4,-1
$ mcal c='sqrt(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=sqrt(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,9,3
2,2,1.414213562
3,,
4,-1,
```

4.94 sqsum 平方和

書式: `sqsum(num1, num2, ...)`

`numi` で与えられた数値の平方和を計算する。NULL 値は無視されるが、全てが NULL 値であれば結果も NULL となる。

利用例

例 1: 基本例

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='sqsum(${v1},${v2},${v3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=sqsum(${v1},${v2},${v3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,14
2,-5,2,1,30
3,1,,3,10
4,,,,
```

例 2: ワイルドカードを利用した例

`v` から始まる項目 (`v1,v2,v3`) をワイルドカード「`v*`」によって指定している。

```
$ mcal c='sqsum(${v*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgcal a=rsl c=sqsum(${v*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,1,2,3,14
2,-5,2,1,30
3,1,,3,10
4,,,,
```

4.95 sum 合計

書式: `sum(num1, num2, ...)`

`numi` で与えられた数値を全て合計する。NULL 値は無視されるが、全てが NULL 値であれば結果も NULL となる。

利用例

例 1: 基本例

```
$ more dat1.csv
id,v1,v2,v3
1,1,2,3
2,-5,2,1
3,1,,3
4,,,
$ mcal c='sum(${v1},${v2},${v3})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=sum(${v1},${v2},${v3}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,v1,v2,v3,rsl
1,1,2,3,6
2,-5,2,1,-2
3,1,,3,4
4,,,,
```

例 2: ワイルドカードを利用した例

v から始まる項目 (v1,v2,v3) をワイルドカード「v*」によって指定している。

```
$ mcal c='sum(${v*})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c=sum(${v*}) i=dat1.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,1,2,3,6
2,-5,2,1,-2
3,1,,3,4
4,,,,
```

4.96 tan タンジェント

書式: $\tan(r)$

ラジアン r に対するタンジェントを計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,0.785398
2,1.047197
3,
4,3.141593
$ mcal c='tan(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=tan(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,0.785398,0.9999996732
2,1.047197,1.732048603
3,,
4,3.141593,3.464102066e-07
```

4.97 tanh 双曲線逆正接

書式: $\tanh(r)$

双曲線逆正接 (ハイパーボリック タンジェント) を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,val
1,3.141592
2,-1.047197
3,
4,6.283185
$ mcal c='tanh(${val})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=tanh(${val}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,val,rsl
1,3.141592,0.9962720714
2,-1.047197,-0.7807142201
3,,
4,6.283185,0.9999930253
```


4.98 time 時分秒

書式: `time(time)`

時刻 *time* から時分秒を 6 桁固定長文字列として取り出す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='time(${time})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=time(${time}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,rsl
1,20000101000000,000000
2,20121021111213,111213
3,,
4,19770812122212,122212
```

4.99 today 本日の日付

書式: today()

本日の日付を日付型で返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id
1
2
$ mcal c='today()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=today() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,20160630
2,20160630
```

4.100 tolower 小文字変換

文字列をすべて小文字に変更する。アルファベット 26 文字以外の文字については何の影響もない。

利用例

例 1: 基本例

str 項目のアルファベット大文字を全て小文字に変換する。

```
$ more dat1.csv
id,str
1,ABC
2,aB$12!Cd
3,
4,cBA
$ mcal c='tolower(${str})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=tolower(${str}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,ABC,abc
2,aB$12!Cd,ab$12!cd
3,,
4,cBA,cba
```

4.101 top 先頭行

書式: top()

先頭行であれば真を、そうでなければ偽を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
val
1
2
3
4
$ mcal c='top()' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=top() i=dat1.csv o=rsl1.csv
$ more rsl1.csv
val,rsl
1,1
2,0
3,0
4,0
```

例 2: 累計計算の例

```
$ mcal c='if(top(),${val},${val}+#{})' a=rsl i=dat1.csv o=rsl2.csv
#END# kgc al a=rsl c='if(top(),${val},${val}+#{})' i=dat1.csv o=rsl2.csv
$ more rsl2.csv
val,rsl
1,1
2,3
3,6
4,10
```

4.102 toupper 大文字変換

文字列をすべて大文字に変更する。アルファベット 26 文字以外の文字については何の影響もない。

利用例

例 1: 基本例

str 項目のアルファベット小文字を全て大文字に変換する。

```
$ more dat1.csv
id,str
1,abc
2,Ab$12!cD
3,
4,Cba
$ mcal c='toupper(${str})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgcal a=rsl c=toupper(${str}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,str,rsl
1,abc,ABC
2,Ab$12!cD,AB$12!CD
3,,
4,Cba,CBA
```

4.103 tseconds 経過秒数

書式: `tseconds(time)`

00:00:00 から時刻 *time* までの経過秒数を計算する。

利用例

例 1: 基本例

```
$ more dat1.csv
id,time
1,000103
2,235959
3,
4,000000
$ mcal c='tseconds(${t{time}})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=tseconds(${t{time}}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,time,rsl
1,000103,63
2,235959,86399
3,,
4,000000,0
```

例 2: 日付が異なっても結果は同じ

```
$ more dat2.csv
id,time
1,20130901000103
2,20130902000103
$ mcal c='tseconds(${t{time}})' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=tseconds(${t{time}}) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20130901000103,63
2,20130902000103,63
```

4.104 uxt UNIX 時刻変換

書式 1: `uxt(date)`

書式 2: `uxt(time)`

書式 3: `uxt2d(num)`

書式 4: `uxt2t(num)`

書式 1,2 では、日付 *date* もしくは時刻 *time* を UNIX 時刻に変換する。逆に書式 3,4 では、UNIX 時刻を日付型もしくは時刻型に変換する。ここで、日付型が与えられたときは、その日の最初の時刻である 00:00:00 として計算される。

利用例

例 1: 基本例

日付型の *date* 項目を `uxt` 関数で UNIX 時刻に変換し、`uxt2d` 関数でまたもとに戻す。

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19700101
$ mcal c='uxt(${date})' a=uxt i=dat1.csv o=rsl1.csv
#END# kgc al a=uxt c=uxt(${date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,uxt
1,20000101,946684800
2,20121021,1350777600
3,,
4,19700101,0
$ mcal c='uxt2d(${uxt})' a=date2 i=rsl1.csv o=rsl2.csv
#END# kgc al a=date2 c=uxt2d(${uxt}) i=rsl1.csv o=rsl2.csv
$ more rsl2.csv
id,date,uxt,date2
1,20000101,946684800,20000101
2,20121021,1350777600,20121021
3,,
4,19700101,0,19700101
```

例 2: 時刻型も同様

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19700101000100
$ mcal c='uxt(${time})' a=uxt i=dat2.csv o=rsl3.csv
#END# kgc al a=uxt c=uxt(${time}) i=dat2.csv o=rsl3.csv
$ more rsl3.csv
id,time,uxt
1,20000101000000,946684800
2,20121021111213,1350817933
3,,
4,19700101000100,60
$ mcal c='uxt2t(${uxt})' a=time2 i=rsl3.csv o=rsl4.csv
#END# kgc al a=time2 c=uxt2t(${uxt}) i=rsl3.csv o=rsl4.csv
$ more rsl4.csv
```

id	time	uxt	time2
1	20000101000000	946684800	20000101000000
2	20121021111213	1350817933	20121021111213
3	,	,	,
4	19700101000100	60	19700101000100

4.105 week 週

書式 1: `week(date)`

書式 2: `week(time)`

書式 3: `week111(date)`

書式 4: `week111(time)`

日付 *date* もしくは時刻 *time* の ISO8601 で規定された週番号を返す。ISO8601 で規定された週番号とは、年の最初の木曜日を含む週をその年の第 1 週と規定している。

一方で `week111` を利用すると、曜日に関係なく 1/1 を第 1 週の第 1 日目と考えて週番号を返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,date
1,20000101
1,20000102
1,20000103
1,20000104
1,20000105
1,20000106
1,20000107
1,20000108
1,20000109
2,20121021
3,
4,19770812
$ mcal c='week(${date})' a=rs1 i=dat1.csv o=rs11.csv
#END# kgc al a=rs1 c=week(${date}) i=dat1.csv o=rs11.csv
$ more rs11.csv
id,date,rs1
1,20000101,52
1,20000102,52
1,20000103,1
1,20000104,1
1,20000105,1
1,20000106,1
1,20000107,1
1,20000108,1
1,20000109,1
2,20121021,42
3,,
4,19770812,32
```

例 2: 時刻型でも可能

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='week(${time})' a=rs1 i=dat2.csv o=rs12.csv
#END# kgc al a=rs1 c=week(${time}) i=dat2.csv o=rs12.csv
$ more rs12.csv
id,time,rs1
1,20000101000000,52
```

2,20121021111213,42
3,,
4,19770812122212,32

4.106 year 西暦年

書式 1: year(*date*)

書式 2: year(*time*)

書式 3: years(*date*)

書式 4: years(*time*)

日付 *date* もしくは時刻 *time* から西暦年を取り出す。書式 1,2 は数値として返し、書式 3,4 は文字列として返す。

利用例

例 1: 基本例

```
$ more dat1.csv
id,date
1,20000101
2,20121021
3,
4,19770812
$ mcal c='year($d{date})' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=year($d{date}) i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,date,rsl
1,20000101,2000
2,20121021,2012
3,,
4,19770812,1977
```

例 2: 時刻型でも可能

```
$ more dat2.csv
id,time
1,20000101000000
2,20121021111213
3,
4,19770812122212
$ mcal c='year($t{time})' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=year($t{time}) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,time,rsl
1,20000101000000,2000
2,20121021111213,2012
3,,
4,19770812122212,1977
```

4.107 型変換

書式:s2n(str), n2s(num), n2b(num), s2n(str), s2d(str), s2t(str) s2b(str), d2s(date), d2t(date), t2s(time), t2d(time), b2n(bool), b2s(bool)

型変換を行う関数群。mcal では型の自動変換は行わないため、必要なときはユーザが明示的に型変換を指定しなければならない。

n2b(s2b) 関数は、1("1") を真に 0("0") を偽に、それ以外を NULL 値に変換する。b2n(b2s) 関数は、真は 1("1") に偽は 0("0") に変換する。

d2t 関数は、日付型を時刻型に変換するが、そのとき時刻として 12:00:00 を自動的に補完する。d2s 関数は、8 文字固定長文字列 ("yyyymmdd") に変換し、t2s 関数は 14 文字固定長文字列 ("yyyymmddHHMMSS") に変換する。日付/時刻についての詳細は「[4.13 日付型と時刻型](#)」を参照のこと。

利用例

例 1: 乱数の固定長化

1 から 9999 の整数乱数を発生させ、4 桁固定長で出力する。fixlen 関数は整数型のデータ (ここでは randi の結果) には対応していないので、n2s 関数で文字列型に変換する必要がある。

```
$ more dat1.csv
id
1
2
3
4
$ mcal c='fixlen(n2s(randi(1,9999,11))),4,"R","0")' a=rsl i=dat1.csv o=rsl1.csv
#END# kgc al a=rsl c=fixlen(n2s(randi(1,9999,11))),4,"R","0") i=dat1.csv o=rsl1.csv
$ more rsl1.csv
id,rsl
1,1803
2,0684
3,0195
4,6647
```

例 2: 真偽パターン

項目 v1,v2,v3 が 10 異常かどうかを判定し、01 のパターンを出力する。

```
$ more dat2.csv
id,v1,v2,v3
1,10,5,7
2,5,12,11
3,3,6,2
4,14,16,11
$ mcal c='cat("",b2s(${v1}>=10),b2s(${v2}>=10),b2s(${v3}>=10))' a=rsl i=dat2.csv o=rsl2.csv
#END# kgc al a=rsl c=cat("",b2s(${v1}>=10),b2s(${v2}>=10),b2s(${v3}>=10)) i=dat2.csv o=rsl2.csv
$ more rsl2.csv
id,v1,v2,v3,rsl
1,10,5,7,100
2,5,12,11,011
3,3,6,2,000
4,14,16,11,111
```